



Escuela Técnica Superior
de Ingeniería Informática

Grado en Ingeniería de la Ciberseguridad

Curso 2025-2026

Trabajo Fin de Grado

**FRAMEWORK ABIERTO DE DESARROLLO Y
GESTIÓN DE LKMS EN LINUX PARA LA
EMULACIÓN CONTROLADA DE TÉCNICAS DE
ROOTKIT**

Autor: Saúl Fernández García

Tutor: Clara Contreras Nevares

Mayo 2026

Agradecimientos

En lo personal,

A mi hermano Adrián, a mis padres Agus y Carlos, a mis abuelos, a Petry, y a los pequeños Cody, Lola, Cooper y Pluto.

Gracias por ser parte de mi vida.

En lo académico,

A mi tutora de TFG Clara Contreras Nevares, gracias por su paciencia conmigo y dirección en este proyecto. También, gracias a mis profesores Marta Beltrán Pardo, María Isabel González Vasco, Isabel Bazaga Fernández, Raúl Martín Santamaría, Sergio Pérez Peló, David Concha Gómez, Sofía Bayona Beriso, César Cáceres Taladriz, Alfonso Fernández Timón, Nicolás Hernán Rodríguez Uribe, María del Soto Montalvo Herranz, Ángel Serrano Sánchez de León, Lidia Andrés Espinal, Gema Gutiérrez Peña, Jesús Sánchez-Oro Calvo, Dave Towey, Anthony Graham Bellotti, y Fiseha Berhanu Tesema.

Gracias por haber sido fuente de inspiración durante la carrera y modelos a seguir en mi ámbito. He tenido el honor de ser alumno de todos ustedes, y me entusiasma saber que labraré mi camino en ciberseguridad con sus enseñanzas.

Resumen

El kernel de Linux está integrado por módulos que componen su funcionalidad. Entre los métodos de *blue teaming* para la detección de malware a nivel de este kernel, destacan la detección de anomalías en módulos y la manipulación de estructuras del kernel. No obstante, actualmente se carece de una solución *open-source* para desarrollar *Loadable Kernel Modules* y a su vez gestionar la ejecución de su funcionalidad. Este proyecto presenta una herramienta que estandariza el desarrollo de estos módulos mediante su propio framework y facilita su ejecución controlada, permitiendo a los usuarios orquestrar en el espacio de kernel de Linux actividad y comportamientos personalizados, actuando así como un primer esfuerzo de código abierto para facilitar la investigación de malware en entornos privilegiados desde el punto de vista de módulos estandarizados, fácilmente gestionables, y compartibles entre la comunidad de desarrollo.

Palabras clave:

- Linux
- Kernel
- LKM
- Linux Kernel Module
- Loadable Kernel Module
- Development Framework
- Ciberseguridad
- Rootkit

Índice de contenidos

Índice de figuras	11
Índice de códigos	13
1. Introducción	15
2. Objetivos y Estructura del Documento	16
2.1. Objetivos del Trabajo de Fin de Grado	16
2.2. Estructura del documento	16
3. Metodología	18
3.1. Metodología de gestión: Kanban	18
3.2. Proceso y evolución del desarrollo	18
4. Estado del Arte y Vanguardia	20
4.1. El Kernel: el Núcleo de un Sistema Operativo	20
4.2. Kernel de Linux y Módulos LKM	21
4.3. Naturaleza y Riesgos de LKMs	22
4.4. Malware en el Kernel de Linux: Conceptualización y Técnicas . .	23
4.5. Rootkits LKM: Importancia, Alcance, y Motivación de este TFG .	24
4.6. Métodos de Detección de Rootkits	25
4.7. Exploración de Herramientas de Vanguardia para la interacción con el Kernel y LKMs	26
4.7.1. LKMs nativos	26
4.7.2. Plantillas	27
4.7.3. DKMS	27
4.7.4. eBPF	27
4.7.5. LSM	28
4.8. Limitaciones en la Actualidad	28
5. Descripción Informática	29
6. Desarrollo de la herramienta	30
6.1. Conocimientos Previos	30

6.1.1.	¿Qué es un LKM, Loadable Kernel Module?	30
6.1.2.	Licencia del Kernel de Linux y Proyectos Derivados	32
6.1.3.	Ciclo de Vida	33
6.1.4.	Compilación: Kbuild y Makefile	33
6.1.5.	Comandos de Consola para LKMs	34
6.1.6.	La clave de un LKM: <code>struct module</code>	34
6.1.7.	Refcount y Módulos	35
6.1.8.	Linux Lists: <code>list.h</code>	36
6.1.9.	Devolviendo información a espacio de usuario: <code>debugfs</code>	39
6.1.10.	Asignación de Memoria en Kernel	43
6.1.11.	Devolución de errores en Kernel	44
7.	Desarrollo de un <i>Check</i> personalizado	45
7.1.	Código mínimo en <code>.c</code> necesario	45
7.2.	Código mínimo del Kbuild necesario	48
8.	Evaluación	49
8.1.	Ejemplo de ciclo de vida	49
8.1.1.	Recomendaciones y entorno	49
8.1.2.	Ejemplo de uso	50
9.	Limitaciones	55
10.	Conclusiones y trabajos futuros	56
10.1.	Conclusiones	56
10.2.	Trabajos Futuros	57
10.2.1.	Arquitectura del núcleo	57
10.2.2.	Ejecución de <i>checks</i>	57
10.2.3.	Interfaz de usuario	57
10.2.4.	Compilación y configuración	58
10.2.5.	ABI y retrocompatibilidad	58
10.2.6.	Documentación y estandarización	58
	Bibliografía	58
	Apéndices	63
A.	Estructura del Proyecto	65
A.0.1.	Organización	65
A.0.2.	Flujo lógico de información	66
A.0.3.	Desarrollo de propia infraestructura de plugins en C	67
A.1.	El núcleo del programa: <code>core</code>	68
A.1.1.	<code>core.c</code>	68

A.1.2. <code>core_debugfs.c</code>	69
A.1.3. <code>core_internal.h</code>	71
A.2. La ABI del programa: <code>include/lkm_check.h</code>	71
A.3. La carpeta <code>checks</code> y sus contenidos	72
B. Código	73
B.1. <i>Check</i> B: <code>check_b</code>	73

Índice de figuras

3.1. Tablero Kanban para el proyecto	19
4.1. Estructura del código fuente del kernel de Linux (fuente: [1]) . . .	21
6.1. Ejecución de <code>lsmod</code> con LKMs relacionados con el proyecto. . . .	36
6.2. Llamadas a syscall <code>read</code> por el comando <code>cat</code>	40
8.1. Puesta a prueba: compilación del núcleo y <i>checks</i>	50
8.2. Puesta a prueba: ubicación de archivos <code>.ko</code> tras compilación. . . .	51
8.3. Puesta a prueba: <code>modinfo</code> de núcleo y <i>checks</i> de ejemplo.	51
8.4. Puesta a prueba: invocación de <code>modprobe</code> sobre núcleo y <i>checks</i> para inserción y visualización de mensajes por buffer de kernel. . .	52
8.5. Puesta a prueba: invocación de <code>modprobe</code> sobre un <i>check</i> antes que el núcleo.	52
8.6. Puesta a prueba: comprobación con <code>ls</code> de archivos virtuales crea- dos en <code>debugfs</code>	53
8.7. Puesta a prueba: uso de <code>available</code> para visualizar <i>checks</i> dispo- nibles.	53
8.8. Puesta a prueba: uso de <code>add</code> para añadir <i>checks</i> y <code>selected</code> para comprobar su selección.	54
8.9. Puesta a prueba: uso de <code>results</code> para obtención de resultados de ejecución de funcionalidad de los <i>checks</i> seleccionados.	54
8.10. Puesta a prueba: uso de <code>remove</code> para la desección de <i>checks</i> de la lista de seleccionados.	54
8.11. Puesta a prueba: uso de <code>addall</code> y <code>empty</code> para seleccionar y deselec- cionar todos los <i>checks</i> disponibles.	54
8.12. Puesta a prueba: retirada de kernel de LKMs <i>core</i> y <i>checks</i>	54
A.1. Estructura del proyecto desarrollado	66
A.2. Flujo lógico de información	67

Índice de códigos

6.1.	Macros de metadatos del módulo núcleo	32
6.2.	Cabecera SPDX y licencia GPL	33
6.3.	Declaración de listas y nodos	37
6.4.	Ejemplo comando adición y escritura de checks	38
6.5.	Función core_debugfs_init y debugfs	41
6.6.	Funciones para operación addall	41
6.7.	Funciones para operación available	42
7.1.	check_b: cabeceras y struct lkm_check	46
7.2.	check_b: funciones __init y __exit	47
7.3.	check_b: funcionalidad de enumeración de procesos	47
7.4.	Código mínimo kbuild para ejemplo check_b	48
B.1.	Código completo en C para ejemplo check_b	73

1

Introducción

El kernel de Linux, núcleo del sistema operativo y nexa entre software y hardware, cuenta con una arquitectura modular que permite extender su funcionalidad mediante módulos de kernel, *Loadable Kernel Modules*. Estos componentes, al ejecutarse con máximo privilegio y tener acceso a todas las estructuras y procesos del kernel, son del interés de cibercriminales y otros atacantes a la hora de desarrollar malware por facilitar la persistencia y manipulación precisa, aunque delicada, del sistema.

Si bien existen métodos y planteamientos para la detección de actividad maliciosa a nivel de kernel, no se identifican herramientas de código abierto que permitan la emulación controlada de comportamientos maliciosos en este nivel.

Este Trabajo de Fin de Grado presenta una solución que estandariza el desarrollo de módulos del kernel de Linux y orquesta su funcionamiento, siendo una herramienta *open-source*, escalable y flexible que permite a los usuarios desarrollar sus propios comportamientos a nivel de kernel.

2

Objetivos y Estructura del Documento

En este capítulo se desarrollarán los objetivos principales de este proyecto así como la estructura del documento.

2.1. Objetivos del Trabajo de Fin de Grado

Este TFG toma como motivación facilitar la investigación en la detección de rootkits y sus comportamientos en el kernel de Linux. Para ello, propone un marco de trabajo flexible y escalable que permita gestionar con sencillez los escenarios personalizados que se deseen en el kernel.

Con este proyecto se busca que el usuario pueda centrarse en el desarrollo de comportamientos y delegar en una herramienta el control de ejecución. Igualmente, que el marco de trabajo a definir también pueda extrapolarse a ámbitos más allá de *blue teaming*, pudiendo contribuir en los esfuerzos de desarrollo de la comunidad de Linux.

2.2. Estructura del documento

El documento se organizará en múltiples capítulos. Primero, en el capítulo 3 se explicarán los procesos seguidos para la investigación del proyecto y el desarrollo de la solución. Después, en el capítulo 4 se presentará al lector una revisión de la tecnología de vanguardia en cuanto al desarrollo en el kernel de Linux, los focos principales de malware en dicho entorno, una exploración de herramientas que permiten interacción entre kernel y módulos de este, y una breve conclusión de las limitaciones actuales.

Con el estado del arte completado, en el capítulo 5 se presentan los recursos

informáticos empleados para completar este proyecto, y posteriormente en 6 se introducirán conocimientos previos de interés para la comprensión de la herramienta desarrollada y sus mecanismos.

El capítulo 7 analizará un caso de uso completo de la herramienta, 8 un uso completo de las funciones desarrolladas, y 9 las limitaciones observadas. Finalmente, en 10 se dará cierre al trabajo con reflexiones de los objetivos conseguidos y propuestas de trabajos a futuro y líneas de investigación a seguir con la herramienta desarrollada como base.

Adicionalmente, se incluyen dos anexos con información adicional. Por un lado, el apéndice A incluye una documentación del proyecto y su estructura. Por otro lado, apéndice B incluye archivos de código referenciados en el documento al completo y sin cortes.

3

Metodología

Este capítulo explorará el marco de trabajo seguido para la organización del proyecto, el planteamiento de desarrollo y su evolución para la creación de la solución propuesta, y una mención de los recursos utilizados.

3.1. Metodología de gestión: Kanban

Para la organización del trabajo se utilizó la metodología Kanban debido a su enfoque en la visualización del flujo de tareas y la limitación del trabajo en curso (*Work In Progress*, WIP), lo cual resulta adecuado en proyectos desarrollados por un único autor. Se definió un tablero compuesto por cuatro columnas: *To Do*, *Doing*, *Done*, *Discarded*. Las tareas se registraban inicialmente y se añadían a *To Do*. Posteriormente, se trasladaban a *Doing* cuando se comenzaba su implementación, y por último se movían *Done* una vez finalizadas, o a *Discarded* en caso de ser descartadas. Esta última columna se utilizó como un registro de tareas no implementadas, incluyendo la justificación de su descarte. Una captura de la apariencia de este tablero se incluye en [3.1](#).

Este flujo permitió una gestión flexible e iterativa del desarrollo, facilitando la adaptación a cambios durante la implementación.

3.2. Proceso y evolución del desarrollo

El proceso de desarrollo comenzó con el estudio del desarrollo de módulos en el kernel de Linux, así como los mecanismos utilizados habitualmente en entornos de bajo nivel para la implementación de rootkits y malware en espacio de kernel. A partir de este estudio inicial, se desarrollaron prototipos de `Loadable Kernel`

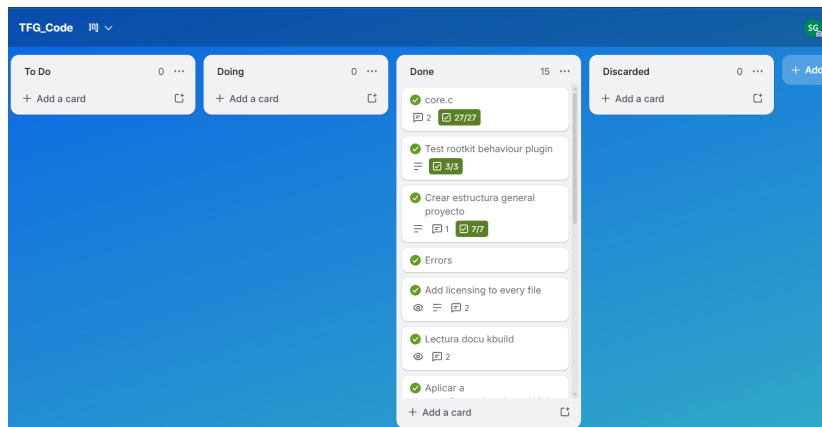


Figura 3.1: Tablero Kanban para el proyecto

Modules (LKMs) con el objetivo de comprender su comportamiento e integración dentro del sistema.

Tras esta fase inicial de prototipado de LKMs de uso general, se progresó hacia la construcción de un módulo principal (*core*) modular basado en complementos que expandiesen e implementasen funcionalidad (*checks*). Las tareas incluyeron la lectura de documentación oficial del kernel de Linux, navegación y análisis del propio código fuente del kernel, revisión de literatura técnica, diseño de la arquitectura del sistema, implementación de prototipos, así como refactorización y ampliación progresiva de capacidades.

4

Estado del Arte y Vanguardia

El núcleo del sistema operativo de Linux, el kernel, es un espacio privilegiado cuyas funcionalidades se pueden extender mediante módulos “LKM”. No obstante, estos módulos son también una oportunidad para atacantes para introducir malware. En este capítulo, se explorarán el kernel de Linux, sus módulos, malware asociados tipo rootkit, detección de estos, y herramientas y recursos para interacción gestionada kernel-LKMs.

4.1. El Kernel: el Núcleo de un Sistema Operativo

Todos los sistemas operativos, como Linux, Windows, o MacOS, ofrecen servicios como la gestión de archivos, conexiones a red, gestión de memoria, gestión de mensajes, funciones de seguridad... y contienen una pieza clave: el kernel (“núcleo”, en inglés). El kernel es una parte del sistema operativo que se ejecuta con privilegios, o en espacio de kernel / espacio privilegiado / *kernel space*; a diferencia del espacio de usuario / *userspace*, que goza de menos privilegios que el espacio de kernel [2, 1. Overview of Operating Systems and Kernels]. Esta capacidad de privilegio permite controlar qué servicios o funciones del sistema operativo se pueden emplear, aunque cómo se gestiona este nivel de privilegio y qué se considera una acción privilegiada depende del tipo de kernel.

Dos de los enfoques de arquitecturas de kernel más representativos son el microkernel y el kernel monolítico [2, 1. Linux Versus Classic Unix Kernels].

Los microkernels se caracterizan por albergar únicamente mecanismos básicos en espacio privilegiado, como planificación, gestión de memoria, y comunicación entre procesos, delegando el resto de funciones del sistema operativo al espacio no privilegiado de usuario, donde conviven con otras aplicaciones del usuario.

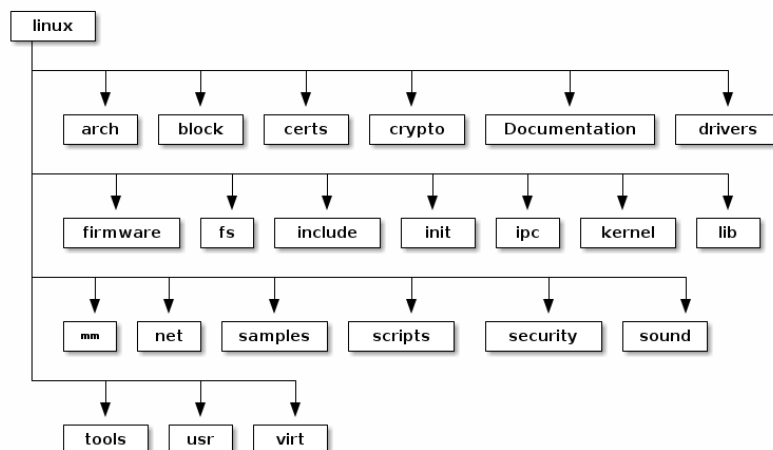


Figura 4.1: Estructura del código fuente del kernel de Linux (fuente: [1])

De forma complementaria, los kernels monolíticos albergan en espacio privilegiado los servicios mínimos que contendría un microkernel y otros como la gestión de conexiones a red o el sistema de archivos, dejando en espacio no privilegiado al resto de aplicaciones del usuario, las cuales pueden solicitar la realización de funciones privilegiadas mediante "syscalls", llamadas al sistema, que ceden la ejecución temporalmente al kernel.

Así, por un lado, un microkernel es intermediario de toda comunicación entre procesos, permitiendo una mayor separación entre componentes y, por ende, una mayor estabilidad ante fallos, aunque el rendimiento puede verse afectado ligeramente. Por otro lado, la estructura de uno monolítico provee, a cambio de menos aislamiento entre componentes (ya que un fallo en nivel privilegiado puede bloquear el resto de ellos), una comunicación más fluida de todos los servicios del sistema operativo.

4.2. Kernel de Linux y Módulos LKM

El código del kernel de Linux, al ser monolítico, contiene mucho más que gestión de memoria o planificación de ejecución, también incluye funciones criptográficas ("crypto"), la gestión del sistema de archivos ("fs"), la implementación de protocolos de red ("net"), o controles de seguridad ("security"), entre muchos otros subsistemas, como esquematiza la figura 4.1.

El kernel se construye como una única imagen ejecutable principal [2, 1. Overview of Operating Systems and Kernels] que se carga en una de las fases de arranque del sistema y en la que pueden incorporarse directamente muchos de estos subsistemas (por lo que son *built-in*, integrados). No obstante, como se mencionó previamente, el kernel de Linux es también "modular", y esto permite la extensión

dinámica de sus capacidades mediante los conocidos como módulos de kernel.

Los módulos de kernel (“Loadable Kernel Modules”, abreviados como LKM), son archivos que contienen código compilado que se ejecuta en espacio privilegiado y amplían las funciones del kernel [2, 17. Modules]. Identificados en Linux con la extensión “.ko” (“kernel object”), pueden cargarse y descargarse dinámicamente sin necesidad de recompilar el kernel ni reiniciar el sistema siempre y cuando no estén siendo utilizados activamente. Al gozar de los mismos privilegios que el resto del kernel, proveen tanto flexibilidad como riesgos potenciales en cuanto a estabilidad y seguridad.

4.3. Naturaleza y Riesgos de LKMs

Los LKMs del kernel de Linux se caracterizan por tener una pieza fundamental: una función de inicialización, que se ejecuta cuando se carga o inserta el módulo y permite preparar los recursos vinculados. Además, si el módulo es descargable, debe incluir también otra función de finalización, que se ejecuta cuando se retira y sirve para una salida segura y limpieza de recursos [2, 17. Modules]. Estos LKMs pueden ser tanto ciertos subsistemas opcionales del kernel (p. ej. drivers para hardware), como funcionalidades completamente nuevas. Independientemente, todo LKM debe definir una función de inicio y una de salida si se quiere permitir su descarga dinámica.

Al ejecutarse en espacio privilegiado y operar con los mismos privilegios que el resto del código del kernel, los LKMs introducen consideraciones críticas tanto en términos de estabilidad como de seguridad.

La programación de un LKM requiere usar adecuadamente las APIs internas del kernel y respetar los mecanismos de sincronización, gestión de memoria, y registro de subsistemas, tal y como se refleja en la documentación oficial del kernel de Linux [3]. Un uso inadecuado de estas interfaces puede provocar errores graves, efectos en cadena, bloqueos de recursos, o incluso un *kernel panic* [4], mecanismo mediante el cual el kernel detiene la ejecución del sistema al detectar un error crítico que le pone en peligro.

Por otra parte, el elevado nivel de privilegio del que disponen los módulos permite un control preciso del comportamiento interno del sistema [5, Section 2.3.1]. Pese a que facilita la extensión legítima del kernel, también ha sido y sigue siendo aprovechada por actores maliciosos para el desarrollo de malware como rootkits en espacio de kernel, ocultación de procesos maliciosos, o manipulación del sistema. Aunque no solamente entran en juego los LKMs maliciosos, también son igual de peligrosos los LKMs con vulnerabilidades.

4.4. Malware en el Kernel de Linux: Conceptualización y Técnicas

Uno de los objetivos más relevantes para un cibercriminal es la obtención del más alto privilegio en un sistema. Este nivel de acceso facilita otras acciones como persistencia en el sistema comprometido, ocultación de su presencia, eliminación de evidencias de actividad maliciosa, o incluso movimiento lateral hacia otras víctimas. Los programas que consiguen estos objetivos se conocen como “rootkits”.

Originalmente, los “rootkits” eran “kits” de herramientas diseñados para permitir a un atacante mantener acceso con privilegios de “root” a sistemas comprometidos, principalmente mediante técnicas de ocultación y evasión de detección. Con la evolución del malware, el término ha ampliado su significado para abarcar software tanto en espacio de usuario como en espacio de kernel, con o sin privilegios, que se centran en ocultar su propia existencia o la de otro software, conexiones a servidores de comando y control (C2) del atacante, u otras piezas del sistema, tal y como recoge el marco MITRE ATT&CK [6]. No obstante, los rootkits en espacio de kernel son más sofisticados debido a su mayor nivel de control sobre el sistema.

Existen múltiples técnicas que emplean los rootkits para estos objetivos y que facilitan su categorización, como indican M. Nadim et al [7]. Entre ellas, destacan:

- **System Service Hijacking:** modificación de las referencias a syscalls almacenadas en la “system call table” o “system service descriptor table” con código malicioso, reemplazo de la tabla de syscalls al completo, o modificación del código de las syscalls mediante instrucciones de salto a código malicioso.
- **Dynamic Kernel Object Hooking:** modificación de las funciones del sistema de archivos. Cuando se interactúa con archivos, syscalls invocan funciones de acceso a estos, como lectura o escritura, referenciadas por objetos de kernel como `file_operations`. Un rootkit puede modificar, en vez de la syscall, la función de acceso al archivo alterando el puntero almacenado en los objetos de kernel hacia una dirección de memoria con código malicioso.
- **DKOM - Direct Kernel Object Manipulation:** manipulación directa de las estructuras de kernel. Por ejemplo, en la tabla de procesos (task list), un proceso malicioso podría eliminarse de dicha lista y así hacerse invisible a otros procesos (al menos en este objeto de kernel), aunque una mala alteración de estas estructuras pueden desencadenar en *kernel panic*.
- **Dynamic Linker Hijacking:** redirección de librerías de código que solicita y necesita un programa antes de su ejecución hacia librerías maliciosas que el atacante ha creado con el mismo nombre; así, cuando finalmente se ejecute

un programa, durante un período cederá ejecución a un programa malicioso en vez de al esperado [8].

- **eBPF infection:** eBPF es una herramienta que permite ejecutar programas a nivel de kernel pero con verificaciones y restricciones, “adheriéndose” a syscalls. Un programa de eBPF malicioso puede así realizar cambios antes y después de una syscall, manipulando recursos e información [9].

4.5. Rootkits LKM: Importancia, Alcance, y Motivación de este TFG

Un rootkit que consiga establecerse con éxito en el kernel gozará del mismo nivel de privilegio que este, pudiendo interferir en el funcionamiento del resto de componentes [10]. Así, un LKM malicioso tiene la capacidad de solamente necesitar ser cargado una vez en el kernel para poder asegurar persistencia y ocultación.

Como se ha visto, un LKM malicioso puede sabotear el kernel para ocultar otros procesos maliciosos específicos, incluyéndose a sí mismo, mediante la manipulación de entradas en la tabla de procesos, o eliminar del registro de conexiones aquellas que son dirigidas a servidores maliciosos. También, puede añadirse como módulo a cargar automáticamente incluso tras reinicios del sistema. Igualmente, un LKM puede manipular la respuesta que devuelven otros módulos o comandos interceptando syscalls, alterando el comportamiento del sistema.

A pesar de la variedad de acciones que puede realizar un LKM una vez está establecido en el kernel, se sigue enfrentando a múltiples dificultades en su desarrollo y despliegue.

Por un lado, su desarrollo requiere conocimiento específico de programación a nivel de kernel, como sus APIs, estructuras de datos, o gestión de memoria. Consecuentemente, errores en la manipulación maliciosa de estructuras de datos de kernel pueden provocar el pánico de este y bloquear el sistema, haciendo a los LKMs frágiles y arriesgados en su despliegue si su código no es estable o no ha pasado por procesos de control de calidad. Todo esto hace su desarrollo más complejo que el de malware destinado a entornos más flexibles como el espacio de usuario.

Por otro lado, la inserción de LKMs requiere que sea realizado por alguien con privilegios de administrador o equivalente, por lo que el atacante necesitará, por ejemplo, tener éxito con una maniobra de ingeniería social y manipular a un usuario con privilegios, infiltrarse en el sistema aprovechándose de malas configuraciones y elevar privilegios, o realizar un ataque de abrevadero sobre la víctima, entre otras posibilidades.

Finalmente, el desarrollo de LKMs está muy acoplado a la versión de kernel objetivo. Cuando un módulo se va a insertar, se verifica que los símbolos que importa existen en su versión. La inserción de un módulo malicioso en una versión

no compatible se traduce en un malware inútil o con capacidades limitadas [11], o incluso directamente rechazado si opciones `CONFIG_MODVERSIONS` están activadas, que controlan la inserción de acuerdo a la versión que indica el propio LKM [3, Module Versioning]. Incluso en el caso de que se desarrolle un rootkit LKM que adapte su flujo de código y llamadas dependiendo de la versión de kernel del dispositivo en el que se encuentra, esto únicamente eleva la complejidad de su desarrollo y, por ende, recursos dedicados por el atacante.

Independientemente, existen casos ejemplares como:

- **KNARK**: LKM que reemplaza los valores de las direcciones de memoria de syscalls de la tabla de syscalls con otras propias maliciosas o directamente las modifica [12, Section 1.2].
- **Adore-ng**: de código abierto, reemplaza los controladores / handlers del VFS con sus propias rutinas y así se oculta a sí mismo y otros archivos o procesos [13, Other kernel-level rootkits].
- **Diamorphine**: emplea kprobes, una función para debugging, para evadir la restricción de seguridad de no tener acceso al símbolo no exportado `kallsyms_lookup_name`, que permite comprobar el mapeo de direcciones de memoria a nombres de estructuras. Así, se llama a kprobes con la función de lookup para obtener la dirección de la tabla de syscalls, y después se realiza DKOM sustituyendo o modificando las funciones almacenadas [14, Section 3.3.5].
- **Reptile**: no manipula syscalls que devuelven estructuras de datos de kernel. En su lugar se engancha a una función empleada por código de VFS para leer contenidos de directorios, `filldir`, y modifica el valor de retorno a `ENOENT` (“no encontrado”) para ocultar archivos [14, Section 3.3.3].

4.6. Métodos de Detección de Rootkits

Existen múltiples enfoques para la detección de rootkits. Siguiendo MITRE ATT&CK [6], estas se basan principalmente en detectar drivers de kernel anómalos o no autorizados o modificación de módulos de arranque (AN1061); inserción de LKMs anómalos, manipulación de rutas en `/dev`, `/proc`, o asignación de `LD_PRELOAD` a shared objects (AN1062); o ejecución de extensiones de kernel no firmadas, manipulación de daemons de arranque, o hooks a librerías del sistema (AN1063).

Por ejemplo, para evitar que un programa o objeto de kernel sea reemplazado por uno malicioso, se realizan comprobaciones de integridad del este y se compara con copias auténticas o checksums predefinidos, o se comprueba si la dirección de memoria donde se encuentra es la esperada. Una herramienta para ello es LKRG - *Linux Kernel Runtime Guard*, que comprueba en tiempo de ejecución la integridad del kernel de Linux y detecta intentos de explotar vulnerabilidades

[15]. También, se pueden firmar criptográficamente los módulos o directamente restringir la inclusión y retirada de LKMs al sistema de acuerdo a una lista blanca de módulos, como sugieren M. Schmidt et al. [16], en vez de directamente bloquear cualquier inserción de módulos y perder la capacidad de expansión que tiene el kernel.

Al igual que los sistemas de red de detección o prevención de intrusos, la creación de perfiles de comportamiento en kernel permite identificar anomalías que pueden ser indicadores de eventos maliciosos, así como la creación de patrones conocidos de actividad maliciosa [7]. Los indicadores pueden ser accesos a rutas específicas, actividad poco habitual de drivers, modificación de estructuras de datos, comportamiento anómalo en comparación con otros dispositivos o máquinas idénticas, o incluso reglas específicas a la casuística y operativa esperada del dispositivo.

Por otro lado, se puede limitar el alcance de programas de eBPF con otras herramientas como SELinux, AppArmor, o BPF LSM (programa de eBPF), facilitando lógica de seguridad con bajo acoplamiento al kernel mediante hooks a funciones y no código interno del kernel [17, Section 4.3].

4.7. Exploración de Herramientas de Vanguardia para la interacción con el Kernel y LKMs

Para el análisis de los recursos existentes principales para la gestión de LKMs, se explorarán las capacidades de estandarización de desarrollo de LKMs, ABIs para el desarrollo, y orquestación y gestión de ejecución. Esto evidenciará las carencias existentes para facilitar la detección de LKMs maliciosos mediante su emulación.

4.7.1. LKMs nativos

Linux, de forma nativa, permite cargar módulos, descargarlos, exportar símbolos para ser empleados por otros, así como gestionar de forma automática las dependencias entre módulos. Para ello, provee de comandos como `insmod`, para la inserción; `rmmmod`, para la retirada; `depmod`, para la generación de dependencias y mapeo de alias; y `modprobe`, para la resolución automática de dependencias durante la carga y descarga de módulos.

Además, las versiones más recientes del kernel de Linux ofrecen abundantes macros para el desarrollo de LKMs, permitiendo (y requiriendo) especificar la función de entrada “init”, de salida “exit”, información respecto al módulo, símbolos especiales disponibles, etc. [2, 17. Modules].

Sin embargo, no existe una estructura común obligatoria para los módulos, estandarización de su ciclo de vida más allá de init y exit, o gestión de ejecución controlada. Los módulos pueden ser insertados en cualquier momento (siempre y cuando sean compatibles con la versión de kernel en la cual se insertan), pero no se

dispone de herramientas directas para la ejecución controlada de sus capacidades.

4.7.2. Plantillas

Aunque existen plantillas base para el desarrollo de LKMs, como son los ejemplos de Kaiwan N Billimoria (29 Febrero 2024) en el repositorio vinculado a su libro “Linux Kernel Programming” [18], estas carecen de un estándar para la definición de módulos o de herramientas para su ejecución controlada.

No se han hallado proyectos de código abierto cuyo objetivo sea una estandarización o una ejecución controlada de LKMs.

4.7.3. DKMS

DKMS, Dynamic Kernel Module Support, desarrollado por Dell, es un framework que, mediante el duplicado del código fuente y los binarios compilados de los módulos del kernel, permite montar el kernel e instalar o desinstalar módulos incluso en versiones dispares, recompilando y permitiendo el funcionamiento de estos en nuevas versiones del kernel [19]. Así, se consigue un control de actualizaciones o esfuerzos de retrocompatibilidad sin alterar el árbol de kernel presente.

Se centra en automatizar la instalación, y no requiere una programación específica de los LKMs, ni permite una ejecución controlada de los módulos más allá de cuándo insertarlos y cuándo retirarlos.

4.7.4. eBPF

Extended Berkeley Packet Filter, también conocida como eBPF, es una tecnología que evoluciona del “BSD Packet Filter” original, expandiendo capacidades y permitiendo el desarrollo de programas que pueden ser cargados en contextos privilegiados, como el kernel de Linux, mediante una ejecución sandboxed (con verificaciones y restricciones de capacidades) [20, Chapter 1]. De esta forma, se permite ampliar las funcionalidades del kernel sin necesitar un recompilado del código fuente entero.

Para ello, los programas basados en eBPF se ejecutan cuando un proceso invoca un hook como puede ser una syscall, función de init o exit de un LKM, tracepoints de kernel, eventos de red, o cualquier otra situación (tanto en espacio kernel como de usuario) que el usuario defina [20, Chapter 2]. Esta ejecución basada en “adherirse” a eventos permite una gran monitorización del sistema sin necesidad de programar a nivel de kernel, modificar aplicaciones existentes, o reiniciar el sistema.

Pese a que eBPF establece un estándar para el desarrollo de sus programas y su interacción controlada con el sistema operativo, no define un framework de desarrollo de LKMs. De hecho, el enfoque y objetivo de eBPF es precisamente evadir ciertas propiedades e inconvenientes de los LKMs, como el acceso sin restricciones a las piezas clave del kernel o la capacidad real de bloquear el kernel

(*kernel panic*) ante un error crítico causado por el módulo o sus interacciones.

4.7.5. LSM

El proyecto Linux Security Modules, LSM, es un framework para el desarrollo de comprobaciones de seguridad, *hooks*, que conectar con nuevas extensiones del kernel. Estas comprobaciones se centran en control de acceso, como consiguen los LSMs “AppArmor” [21] o “SELinux” [22], brindando funcionalidad como mandatory access control (MAC) a decisiones de seguridad en el sistema. Específicamente, LSM lo logra mediante una API para políticas de seguridad [23].

Pese a incluir en su nombre *modules* y a que el desarrollo de estos hooks sí establece un set de funciones específicas para interactuar con inodes, tareas, y archivos, los LSM no es ni define el desarrollo de LKMs, sino que complementa funcionalidades de seguridad que el kernel, y, por ende, cualquier LKM, pueda requerir [24] [25].

4.8. Limitaciones en la Actualidad

Como se ha podido observar en la sección anterior, por un lado, los métodos principales para la detección de LKMs maliciosos se basan en la detección de anomalías. Por otro lado, entre las herramientas y recursos actuales investigados, no existe un framework de código abierto que formalice una interfaz de desarrollo de LKMs, permita un desarrollo estandarizado, y además centralice la gestión, selección, y ejecución de LKMs personalizados.

5

Descripción Informática

La herramienta sigue una estructura modular basada en los módulos cargables del kernel (LKM). Sus componentes principales son:

- **core**: módulo LKM principal, responsable del descubrimiento y gestión de *checks* así como de la creación de mecanismos que permitan interacción con ellos. A lo largo del documento también se referirá a este como “gestor” o “núcleo”.
- **check**: módulo LKM gestionado por y dependiente del *core* y que implementa una funcionalidad o comportamiento específico dentro del kernel. A lo largo del documento también se referirá a ellos como “comportamiento” o “plugin”.
- **ABI**: interfaz binaria definida en `include/lkm_check.h` que define la interfaz y los requisitos que deben cumplir los *checks* para registrarse e interactuar con el núcleo.

La interacción con el usuario se realiza mediante el sistema de archivos virtual `debugfs`, a través del cual es posible consultar el estado del sistema, seleccionar funcionalidades y ejecutar los *checks* registrados.

Todo el desarrollo del código se hizo en una máquina virtual de Ubuntu 22.04 LTS (Jammy Jellyfish) con una versión del kernel de Linux 6.8.0-94-generic. El código está escrito en C, se compila mediante tecnología Makefile y Kbuild, y emplea el sistema de archivos virtual de Linux “debugfs”. Por último, el código fuente se gestionó mediante Git y se alojó en GitHub para facilitar su control de versiones y distribución.

En el apéndice [A](#) se documentan con mayor detalle la estructura interna del proyecto, la interacción entre sus componentes y las funciones desarrolladas.

6

Desarrollo de la herramienta

Este capítulo explicará los recursos y mecanismos relevantes para la herramienta desarrollada, justificando a su vez las decisiones tomadas y cómo se reflejan en el proyecto.

6.1. Conocimientos Previos

Esta sección explicará los conceptos necesarios para una comprensión de las tecnologías, mecanismos, y convenciones primordiales para la programación en kernel de Linux, desarrollo de LKMs, y la creación de la solución propuesta. A medida que se profundiza en los conceptos, se elaborará en cómo estos se manifiestan en el proyecto desarrollado.

6.1.1. ¿Qué es un LKM, Loadable Kernel Module?

La definición de Peter Jay Salzman et al. en “The Linux Kernel Module Programming Guide” [11] para LKMs es congruente con todo explicado previamente:

Un módulo de kernel de Linux se define precisamente como un fragmento de código capaz de carga y descarga dinámica dentro del kernel a demanda. Estos módulos aumentan las capacidades de kernel sin necesitar un reinicio del sistema.

Nótese que en el desarrollo de LKMs, los términos que se emplean son tanto *Linux Kernel Modules* como *Loadable Kernel Modules*. Pese a que el primero denota módulos del kernel de todo tipo, y el segundo aquellos con inserción dinámica, en este trabajo se emplearán ambos indistintamente para referirse a los que se insertan dinámicamente.

El código fuente de estos módulos se basa en programas escritos en C, sus cabeceras, y Makefile (o Kbuild). Al compilarse contra las librerías del kernel, se obtiene un “kernel object” (.ko), un binario que el kernel puede leer y enlazar a su proceso de ejecución.

Existen módulos *built-in*, que se compilan directamente dentro de la imagen del kernel, y *loadable* / cargables, objetivo de este TFG. Se pueden reconocer por estar compilados con una configuración de makefile de `obj-y += ...` (“yes”) y `obj-m += ...` (“module”), respectivamente [26, 3.1 Goal Definitions]. La herramienta desarrollada se compilará con la opción “-m” para simplificar su inserción y retirada sin tener que reiniciar el sistema con cada uso que se le dé. Compilar con éxito dependerá de que se estén empleando las cabeceras del kernel para el que se crea, y esto se basa en la versión.

Versión de Kernel escogida

La herramienta desarrollada emplea las librerías de kernel de versión 6.8.0-94-generic. Esta decisión responde a criterios de estabilidad y disponibilidad de recursos actualizados, evitando el uso de versiones experimentales.

Dado que el kernel de Linux no garantiza estabilidad para módulos entre versiones, la compatibilidad con otras dependerá de los cambios introducidos en la ABI interna. Por ello, se delega en los desarrolladores de módulos que utilicen esta herramienta la responsabilidad de adaptar su código mediante las prácticas habituales, como la detección de versión de kernel y el uso condicional de interfaces.

El comando `shell uname -r` se puede emplear para comprobar la versión del kernel de la máquina en la que se está desarrollando, y se utilizará en el makefile para facilitar su compilación en una máquina Ubuntu 22.04 LTS. Las cabeceras de kernel se almacenan en la ruta `/lib/modules/[versión_de_kernel]/build/`, y una forma básica de compilar el proyecto sería entonces [27]: `$(MAKE) -C /lib/modules/$(shell uname -r)/build M=$(PWD) modules`.

Macros Necesarias

Durante el desarrollo se emplean varios tipos de macros. Todos los módulos vienen definidos, como mínimo, por una función de entrada al código, indicada mediante la macro de registro `module_init(...)` y la macro de función `__init`. Los módulos *built-in* no requieren una función de salida ya que no se descargan, mientras que los *loadable* sí la necesitan, siguiendo macros similares: `module_exit(...)` y `__exit` [2, 17. Modules]. Ejemplos podrían ser `static int __init welcome(void) y module_init(welcome)`, y `static void __exit farewell(void) y module_exit(farewell)`. Los módulos dinámicos requieren ambas funciones para poder preparar sus recursos y, más importante, gestionar su correcta liberación en caso de que sean retirados.

Además, existen macros para metadatos del programa, como el autor: `MODULE_AUTHOR()`, alias: `MODULE_ALIAS()`, o descripción: `MODULE_DESCRIPTION()`. Se verán más adelante dos tipos de macros relevantes en cuanto a versionado y licencia [28, 4. Module macros]: `MODULE_LICENSE()` y `EXPORT_SYMBOL_GPL()`.

El fragmento 6.1 muestra un uso de las macros:

Código 6.1: Macros de metadatos del módulo núcleo

```
MODULE_LICENSE("GPL");  
MODULE_ALIAS("sfgcore");  
MODULE_AUTHOR("SAUL_FERNANDEZ_GARCIA");  
MODULE_DESCRIPTION("Development_version_of_core_for_lkm_  
management.");
```

6.1.2. Licencia del Kernel de Linux y Proyectos Derivados

El kernel de Linux v6.8 está bajo licencia *GNU General Public License v2-only*, también conocida como “GPL-2.0”, que otorga a los usuarios plena libertad para ejecutar, estudiar, compartir, o modificar el software [29, Preamble].

Todo esto es bajo la condición de que si es distribuido (es decir, compartido o vendido), ya sea en formato original o mediante un producto derivado, este también esté bajo la misma licencia, haciendo que un producto derivado distribuido a otros usuarios no pueda ser propietario. Si, en cambio, es para un uso interno privado y no se distribuye a terceros, este puede mantenerse privado. Además, si el código o un producto derivado está patentado, este debe permitir a los destinatarios usarlo libremente bajo la propia licencia GPL-2.0 y sin requerir regalías [29, Terms and Conditions, 7].

La documentación oficial del kernel para las reglas de licencias declara que los módulos cargables dinámicamente requieren la macro `MODULE_LICENSE()` incluida en el código especificando el string correspondiente para su licencia [30, `MODULE_LICENSE`]. Si esta es propietaria y no GPL, ya sea como licencia única o dual, el kernel se rehusará a resolver símbolos que hayan sido exportados por el código fuente del kernel mediante `EXPORT_SYMBOL_GPL()` [31, `MODULE_LICENSE`, Table 1].

Licencia de este TFG y su código: GPL-2.0

Este proyecto, basado en módulos cargables dinámicamente y externos al árbol, se distribuye bajo la misma licencia que el kernel de Linux, “GPL-2.0”, buscando emular la naturaleza de código *open-source*, libre, y accesible que predica y protege el propio kernel de Linux para cualquier usuario que quiera emplearlo.

Esto se podrá comprobar, por ejemplo, en los archivos que componen los módulos mediante la macro mencionada anteriormente: `MODULE_LICENSE("GPL")`. A su vez, siguiendo el estilo de código del kernel [30, 31], se incluye en la cabecera

de todo archivo un identificador de SPDX - *Software Package Data Exchange* para facilitar la identificación de la licencia bajo la que se distribuye, como en el fragmento 6.2.

Código 6.2: Cabecera SPDX y licencia GPL

```
// SPDX-License-Identifier: GPL-2.0
* LKM: core manager
* Copyright (C) 2026 Saul Fernandez Garcia
<https://github.com/saulfernandezgarcia>
[... ]
MODULE_LICENSE("GPL");
```

Desde el núcleo se exportan las funciones `core_register_check()` y `core_unregister_check()`, partes clave de la API interna disponible a los *check* para registrarse y retirarse del sistema. Ya que estas funciones son específicas para el framework desarrollado y la licencia del proyecto es GPL, no se exportan al sistema mediante `EXPORT_SYMBOL`, sino `EXPORT_SYMBOL_GPL()`.

6.1.3. Ciclo de Vida

Así, el ciclo de vida de un LKM cargable / dinámico comienza con comprobaciones de versionado y licencia, función de inicialización, operativa programada en el módulo, y función de finalización. El módulo estará cargado en el kernel hasta que se comande su retirada o lo que dure la sesión. Si el sistema operativo se reinicia o se apaga, en el rearranque no estarán cargados (a diferencia de los *built-in*) a no ser que se configure su carga automática en arranque (en `etcmodules-load.d`) u otro módulo en ese listado o *built-in* comande su inserción [28, 5. Auto-loading modules on system boot].

Este proyecto empleará un LKM dinámico “núcleo” / *core* que se insertará en el kernel. Este núcleo interactúa con una interfaz que especifica cómo programar otros LKMs “plugins” / *checks* de acuerdo al framework. Finalmente, los LKMs “plugins” / *checks* compilados contra el núcleo y las librerías de kernel podrán ser insertados y gestionados por el núcleo.

6.1.4. Compilación: Kbuild y Makefile

La compilación de programas en C se suele automatizar con archivos “Makefile” escritos en el lenguaje Make. Cuando se desea compilar un proyecto, se invoca el comando `make`, que busca un archivo Makefile con instrucciones de construcción [32, 2.3 How make Processes a Makefile]. Al tener capacidad jerárquica y recursiva, un Makefile puede invocar la compilación de subdirectorios (donde se buscará otro Makefile “hijo”), permitiendo gestionar proyectos grandes divididos en subproyectos [32, 5.7 Recursive Use of make].

En el caso del kernel de Linux, el sistema de construcción utiliza Make junto con el sistema de compilación Kbuild [27, 1. Introduction]. Kbuild es un sistema de

compilación basado en Make y específico del kernel, creando nuevas convenciones, reglas y variables para gestionar la envergadura del propio proyecto. En conjunto, Makefile comanda la compilación, y Kbuild define qué componentes del kernel deben compilarse y cómo hacerlo siguiendo las reglas propias del kernel.

En este proyecto, se seguirá el planteamiento del propio kernel para distribuir los Makefile y Kbuild entre el núcleo y los plugins, poseyendo un Makefile “padre”, un Kbuild “padre”, y Kbuilds específicos para cada LKM creado.

6.1.5. Comandos de Consola para LKMs

Existen múltiples comandos básicos para la interacción con LKMs externos al árbol del kernel. Entre ellos, los principales y más usados para este proyecto son [2, 17. Modules: “Installing Modules” to “Loading Modules”]:

- `insmod`: inserción de un LKM específico mediante ruta al archivo `.ko`.
- `rmmmod`: retirada de un LKM específico mediante el nombre del módulo.
- `modprobe`: inserción de un LKM específico mediante nombre del módulo o alias. Resuelve dependencias automáticamente.
- `modprobe -r`: retirada de un LKM específico mediante nombre del módulo o alias, gestionando dependencias.
- `depmod`: genera mapeo de dependencias entre módulos y construye índices de alias (archivos `modules.dep`, `modules.alias`).
- `modinfo`: extrae información del LKM, como versión, autor, licencia, o descripción.
- `lsmod`: enumera los LKMs actualmente cargados en el kernel (contenidos en `/proc/modules`).

Los comandos `depmod` y `modprobe` son particularmente útiles ya que facilitan la organización y resolución de dependencias. Por un lado, `depmod` analiza los módulos y genera las dependencias entre ellos cuando un LKM exporta o requiere símbolos (`modules.dep`), y crea un índice de alias definidos en los módulos (`modules.alias`). Posteriormente, `modprobe` resuelve un alias al nombre del módulo y, a partir de este, obtiene la ruta con la información de las dependencias.

6.1.6. La clave de un LKM: `struct module`

La estructura `struct module` constituye la abstracción central mediante la cual el kernel representa un LKM en memoria. A través de ella, el kernel no solo almacena información descriptiva del módulo, sino que también gestiona su integración, ejecución, y ciclo de vida.

Entre sus campos, se incluyen identificadores básicos como el nombre (`name`), estado (`state`), y versión (`version`), así como punteros a las funciones clave (`init` y `exit`) que definen los puntos de entrada y salida del módulo. Además, incorpora estructuras para la gestión de los símbolos exportados (`syms`) y su resolución, lo que permite la interacción entre módulos [33].

También contiene información sobre dependencias mediante listas enlazadas (`source_list` y `target_list`), y un controlador de referencias (`refcnt`), que permite gestionar el uso activo del módulo en tiempo de ejecución (se profundizará más en estos campos en la sección 6.1.7). Por otro lado, incluye información sobre los parámetros pasados por consola durante su inserción (`kp`) [33].

En conjunto, `struct module` permite al kernel tratar un módulo como una entidad cuya visibilidad, dependencias, carga, ejecución, y descarga están gestionadas con mecanismos internos.

6.1.7. Refcount y Módulos

Cuando varios módulos interactúan entre sí, el kernel debe garantizar que ninguno de ellos sea retirado mientras otros módulos o partes del sistema siguen dependiendo de él. Para ello, gestiona las relaciones mediante dos mecanismos complementarios: una red de dependencias y un sistema de conteo de referencias dinámico.

Por un lado, cuando se ejecuta `insmod` o `modprobe`, el kernel resuelve los símbolos no definidos del módulo mediante aquellos exportados por otros con `EXPORT_SYMBOL`. Durante este proceso se construye una red de dependencias representada mediante listas enlazadas (`source_list` y `target_list`) dentro de `struct module` [33, L565]. Estas listas registran, respectivamente, los módulos que dependen de un módulo determinado y aquellos de los que dicho módulo depende. Esta lógica se implementa en el cargador de módulos (`kernel/module/main.c`), especialmente durante la ejecución de `load_module()` y funciones asociadas a la resolución de símbolos, como `resolve_symbol()`, que finalmente invocan a `add_module_usage()` para registrar dichas relaciones [34, L582, L2831].

Este mecanismo garantiza que un módulo no pueda ser descargado mientras existan otros módulos cargados que dependan de él.

Por otro lado, el kernel emplea un contador de referencias para los módulos, `refcnt` (refcount), implementado como un `atomic_t` dentro de `struct module` [33, L572], para gestionar el uso activo en tiempo de ejecución. Refleja el número de usos activos del módulo y, mientras sea mayor que cero, también impide su descarga inmediata. Este “uso activo” se refiere a situaciones en las que existe código en ejecución o recursos del sistema asociados al módulo.

Algunos ejemplos son descriptores abiertos (por ejemplo, un usuario leyendo un archivo), callbacks de otros subsistemas (como funciones asíncronas), o funciones pertenecientes al módulo que se encuentra en ejecución [28, 5. Understanding module stacking].

```

saul@ubuntu-lkm:/lib/modules/6.8.0-94-generic/sfg$ lsmod | grep sfg
sfgcheck_b          12288  0
sfgcheck_a          12288  0
sfgcore             24576  2 sfgcheck_a,sfgcheck_b
saul@ubuntu-lkm:/lib/modules/6.8.0-94-generic/sfg$

```

Figura 6.1: Ejecución de `lsmod` con LKMs relacionados con el proyecto.

A diferencia del grafo de dependencias, este contador no se incrementa automáticamente durante la resolución de símbolos. En su lugar, debe ser gestionado explícitamente para los módulos mediante las funciones `try_module_get()` y `module_put()`, que incrementan y disminuyen el contador, respectivamente [35] [34, L829, L845]. A estos conceptos se les conoce como *getting* y *putting* [2, 17. Reference Counts]. Por tanto, cuando un módulo o subsistema comienza a utilizar funcionalidad de otro módulo en tiempo de ejecución, debe invocar `try_module_get()` sobre el módulo proveedor, y posteriormente `module_put()` cuando deje de utilizarlo.

Estos dos mecanismos permiten una correcta gestión del ciclo de vida de un módulo cuando interactúa con otros. Si el programa desarrollado necesita símbolos de otros módulos, dichos símbolos deberán exportarse mediante `EXPORT_SYMBOL`. Si el programa mantiene referencias en tiempo de ejecución de otro módulo, será necesario previamente solicitar un aumento de *refcount* del otro módulo y, una vez finalizado su uso, su disminución.

Cabe destacar que ambos mecanismos son independientes. La existencia de dependencias no implica un aumento del contador de referencias. Un módulo puede tener un *refcount* de 0 porque no tiene referencias activas en uso y, sin embargo, no poderse retirar debido a que otros módulos dependen de él. Una forma de comprobar el *refcount* de los módulos insertados y aquellos dependientes es con el comando `lsmod`, como se ilustra en la imagen 6.1.

6.1.8. Linux Lists: `list.h`

El kernel de Linux dispone de su propia implementación de una lista doblemente enlazada y circular (en `<linux/list.h>`) mediante la estructura `struct list_head`. Aquí, se empleará para gestionar los *checks* cargados en el núcleo del programa, y `core.c` gestionará la lógica de manipulación de estas listas.

Existen implementaciones de listas doblemente enlazadas, como la `LinkedList` en Java, que se basan en tres entidades: una estructura que representa la lista [36, L90], otra que representa cada nodo [36, L982], y los datos a almacenar en los nodos. Cada nodo contiene referencias al nodo previo y al siguiente, mientras que la lista mantiene referencias al primer y último elemento, y en los nodos se almacenan los tipos de datos que se quieran listar, aunque estos datos no son conocedores de la lista a la que pertenecen.

En cambio, la implementación del kernel de Linux sigue un enfoque diferente basado en listas intrusivas. En este modelo, no existe una estructura de lista se-

parada de los nodos, sino que cada estructura de datos que desea formar parte de una lista contiene en su interior un campo de tipo `struct list_head` que almacena punteros a otros `struct list_head` de otros datos, por lo que los propios datos a listar saben a qué listas pertenecen. De hecho, una lista se representa mediante `struct list_head`, que contiene punteros al elemento previo `list_head *prev` y al siguiente `list_head *next` [37, L193]. Para inicializar una lista, se emplea la macro `LIST_HEAD(nombre)` [38, L25], que define un nodo especial que apunta a sí mismo actuando como nodo “centinela”, cabeza de la lista, y que permite representar una lista vacía.

Los elementos que forman la lista contienen su propio ítem `struct list_head` embebido. Para insertar un nuevo elemento al inicio de la lista, se llama a la función `list_add()` [38, L167], pasando como primer argumento la dirección del `list_head` del propio nodo a insertar y como segundo argumento la cabeza de la lista. Internamente, esta operación actualizará los punteros `prev` y `next` para mantener la estructura doblemente enlazada circular.

Listas en este TFG

La decisión de escoger esta estructura de datos se basa principalmente en seguir el canon de diseño de listas establecido por el kernel y en que se puedan crear listas sin necesidad de incluir en la definición de cada *check* (`lkm_check`) a cuáles pertenece o cuáles son los siguientes nodos.

En el núcleo, `core.c`, existen dos `structs` que definen los nodos de dos listas distintas: `entry_available` y `entry_selected`. Ambos tipos de nodos contienen un puntero a un *check* (`lkm_check`) y un `list_head` de pertenencia a la lista. Estas dos listas están primero definidas como variables estáticas globales con alcance de fichero, como se muestra en el fragmento 6.3.

Código 6.3: Declaración de listas y nodos

```
static LIST_HEAD(list_available);
static DEFINE_MUTEX(lock_list_available);
static LIST_HEAD(list_selected);
static DEFINE_MUTEX(lock_list_selected);
[...]
struct entry_available{
    struct list_head list;
    struct lkm_check *check;
};
struct entry_selected{
    struct list_head list;
    struct lkm_check *check;
};
```

Los nodos de `list_available` son `entry_available` y los de `list_selected` son `entry_selected`. La primera almacenará los *checks* disponibles, y la segunda

aquellos seleccionados. Creando estos **structs** nodos de cada lista que almacenan referencias a los *checks*, se consigue encapsular en el núcleo la lógica de manipulación de listas y mantener los *checks* ajenos a ella. A su vez, si se quisiesen aumentar las listas o las características de sus nodos, no se interferiría con la lógica y estructura de la ABI de los *checks* (**struct** `lkm_check`).

Las listas son variables de tipo estático para evitar colisiones de símbolos y encapsular la implementación, haciendo que ningún *check* esté involucrado en o manipule directamente las listas a las que pertenece.

Además, toda manipulación de las listas se hará bajo el control de mutex con `lock_list_available` y `lock_list_selected`. Aunque en este proyecto no se crean múltiples hilos explícitamente, en el kernel pueden producirse accesos concurrentes debido a la ejecución en múltiples CPUs o la interacción simultánea desde espacio de usuario a través de mecanismos como debugfs [39, Concurrency considerations], que específicamente se usa en este proyecto. Se decidió no emplear el mecanismo de sincronización RCU (*Read, Copy, Update*) para las listas ya que su uso está recomendado para contextos donde principalmente se leen estructuras y no se modifican [40, RCU Overview].

Por ejemplo, en el proyecto se crean ficheros virtuales mediante la interfaz *debugfs* como `add` (escritura), `remove` (escritura), y `selected` (lectura) para seleccionar y deseleccionar *checks* y ver la lista de aquellos seleccionados. Estos pueden ser accedidos por múltiples procesos en paralelo si la ejecución en múltiples CPUs está activada. Con el comando 6.4 por consola:

Código 6.4: Ejemplo comando adición y escritura de checks

```
echo beep > /sys/kernel/debug/lkmsfg/add & cat /sys/kernel
/debug/lkmsfg/selected
```

Se podría enviar a un CPU la tarea de añadir (`.../add`) el *check* “beep” a la lista de seleccionados, y a otro CPU la de visualizar los *check* seleccionados (`.../selected`), desencadenando dos procesos distintos accediendo a una misma estructura de datos (`list_selected`) de forma concurrente y sin protecciones a no ser que el acceso esté controlado por *locks* como mutex. Un caso más peligroso sería *use-after-free* si se comanda la lectura de la lista de seleccionados y en otro comando la retirada de todos los *check* seleccionados; sin un *lock* se podría acceder a memoria de kernel ya liberada. Por último, en el caso de que otros módulos intentaran emplear las funciones exportadas por el núcleo para el registro y desregistro de *check* (`core_register_check` y `core_unregister_check`), se presentan más situaciones de concurrencia externas al proyecto pero igualmente problemáticas sin un acceso controlado.

Así, los mutex garantizan exclusión mutua en las secciones críticas, evitando condiciones de carrera, inconsistencias en las listas y errores *use-after-free*.

6.1.9. Devolviendo información a espacio de usuario: debugfs

Debugfs es un sistema de archivos que permite a los desarrolladores hacer información de kernel disponible al espacio de usuario [41]. Para ello, se crea un directorio virtual que alberga archivos virtuales (siempre y cuando el kernel se haya compilado con soporte de debugfs). En esencia, estos archivos no son archivos reales almacenados en disco ya que su ciclo de vida se limita a memoria RAM y sirven como interfaz para acceder a información a nivel de kernel.

La función `debugfs_create_file()` crea una entrada en el VFS (`dentry`) junto con su `inode` correspondiente, y asocia a este último la estructura `file_operations` que define su comportamiento [42].

El comportamiento de archivos: `file_operations`

`struct file_operations` (también conocido como `fops`), es un conjunto de operaciones de archivos que implementan el comportamiento de estos [41]. Por ejemplo, se puede crear un archivo virtual llamado “contador” cuyo comportamiento sea que cada vez que se lea con el comando `cat contador` devuelva el número de veces que ha sido leído. Para ello, en sus `fops` en el campo `.open` se crearía una función que mantuviese un contador que aumentase cada vez que se abriese. Para este proyecto, los campos relevantes del `struct` de `fops` son [43, 4. - An introduction to device file operations] [44, L1983]:

- `struct module *owner`: identifica al módulo que acoge la estructura.
- `ssize_t (*read) (struct file *, char __user *, size_t, loff_t *)`: función que realizará una acción de lectura, devolviendo el número de bytes leídos exitosamente o un error.
- `ssize_t (*write) (struct file *, const char __user *, size_t, loff_t *)`: función que realizará una acción de escritura, devolviendo el número de bytes escritos exitosamente o un error.
- `int (*open) (struct inode *, struct file *)`: función destino de syscall `open()`, de apertura de un archivo, devolviendo 0 (éxito) o un error.
- `int (*release) (struct inode *, struct file *)`: función de destino de syscall `close()`, de cierre de un archivo, devolviendo 0 (éxito) o un error.
- `loff_t (*llseek) (struct file *, loff_t, int)`: función que moverá la posición del cursor en el archivo pasado, devolviendo o bien la nueva posición del cursor o bien un valor negativo para indicar error.

Que se llame una función u otra de las definidas en `fops` depende de la syscall del sistema. Un comando que invoque la syscall `read()` una o más veces provocará una apertura del archivo con `fops.open`, después, una o múltiples lecturas con


```

saul@ubuntu-lkm:~/Desktop$ strace -e read cat hi.txt
read(3, "\177ELF\2\1\1\3\0\0\0\0\0\0\3\0-\0\1\0\0\0P\237\2\0\0\0\0...", 832) = 832
read(3, "hi! \n", 131072)          = 7
hi! \n
read(3, "", 131072)                = 0
+++ exited with 0 +++
saul@ubuntu-lkm:~/Desktop$

```

Figura 6.2: Llamadas a syscall `read` por el comando `cat`.

`fops.read`, y por último cierre con `fops.release`. Igualmente, uno que invoque la syscall `write()` provocará una apertura, una escritura con `fops.write`, y un cierre.

Por ejemplo, durante la ejecución del comando `cats` para la lectura de un archivo `hi.txt`, como se observa en la figura 6.2, se llegaron a ejecutar tres syscalls de lectura con `read()`.

La lectura de contenidos: `seq_file`

En el kernel de Linux, para facilitar el devolver información al espacio de usuario a través de archivos virtuales (como `debugfs`) existe la abstracción `seq_file`, que simplifica la interacción con archivos de lectura gestionando buffers e iteración [45], sobre todo en casos donde hay múltiples operaciones de lectura para un mismo acceso a un archivo.

Si se implementase un iterador completo con `seq_file` para navegar los datos durante una secuencia de llamadas a la syscall `read()` sobre el mismo archivo, habría que definir funciones `start()` para inicializar el iterador, `next()` para avanzar sobre los datos, `show()` para devolver output, y `stop()` para finalizar el iterador [45, The iterator interface]. No obstante, en el caso de que se quiera devolver todo el contenido del archivo virtual, se pueden emplear las funciones `single_open()` y `single_release()`, que harán que se invoque una única vez la función `show()` definida [45, The extra-simple version].

Además, ofrece funciones para formatear y mostrar contenido, como `seq_printf()`, que recibe el `seq_file` y el texto a imprimir por pantalla [45, Formatted output].

Sin esta abstracción, implementar correctamente `read()` sobre estructuras dinámicas (como las listas enlazadas) sería complejo, ya que sería necesario gestionar manualmente buffers parciales, *offsets*, y múltiples llamadas a `read()`. Con su uso, se simplifica a una generación de contenido de forma secuencial ajena al estado de iteración o desplazamientos del archivo.

El sistema de archivos y su uso en este TFG

En este TFG, en `core_debugfs.c` se crea un directorio virtual `lkmsfg` donde después se añaden varios archivos virtuales con sus `fops` vinculados (además, se definió para ello la macro `CREATE_FILE()`, que reduce el volumen de código para comprobación de errores). Estos archivos permitirán manipular los *checks*

disponibles, pudiendo ejecutar funciones específicas o retirarlos, por ejemplo.

Código 6.5: Función `core_debugfs_init` y `debugfs`

```
int core_debugfs_init(void){
    lkm_dir = debugfs_create_dir("lkmsfg", NULL);
    [...]
#define CREATE_FILE(name, mode, fops) \
    do{ \
        file = debugfs_create_file(name, mode, lkm_dir, \
            NULL, fops); \
        if(!file) \
            goto err; \
    } while (0)
    CREATE_FILE("available", 0444, &fops_available);
    CREATE_FILE("addall", 0200, &fops_addall);
    [...]
#undef CREATE_FILE
    [...] }
```

En el fragmento de código 6.5 se muestran dos de los archivos virtuales creados: “addall” y “available”, uno dedicado a seleccionar todos los *checks* y otro para mostrar el nombre de los *checks* disponibles. Se explicará el uso de `debugfs`, `fops`, y `seq_file` con ellos, donde el fragmento 6.6 para “addall” mostrará el mecanismo de escritura en listas de *checks* y 6.7 para “available” el sistema específico con funciones *callback* para lectura.

Por un lado, el archivo “addall” tiene vinculado el `fops fops_addall` que define la función `addall_write()`, la cual llama al método `core_addall()` del núcleo que gestiona la adición de todos los *checks* disponibles a la lista `selected`. El valor de retorno de `core_addall()` se propaga en caso de error, pero en condiciones normales el archivo se utiliza como un mecanismo de activación de lógica interna más que como canal de transferencia de datos.

Código 6.6: Funciones para operación `addall`

```
static ssize_t addall_write(struct file* file, const char
    __user *user_buffer,
    size_t size, loff_t *offset){
    int ret = 0;
    ret = core_addall();
    if(ret < 0){
        return ret;
    }
    *offset += size;
    return size;
}
static const struct file_operations fops_addall = {
```

```

    .owner = THIS_MODULE,
    .write = addall_write ,
};

```

Por otro lado, el archivo “available” sí que aprovecha la interfaz de `seq_file` mediante la función `single_open()`. En primer lugar, en `fops_available` se define como campo de `.open` a la función `available_open()`, una que actúa como envoltorio y delega en `single_open()` pasando como parámetro la función personalizada `available_show()`, y como “available” es un archivo virtual destinado a mostrar los *checks* disponibles, esta invoca `core_for_each_available`, un método del núcleo que recorre la lista `list_available` y realiza una acción sobre cada `entry_available` que la compone.

Nótese que el hecho de haber empleado `single_open()` ha requerido que también se empleara `single_release` en `.release`, como se explicó anteriormente.

Código 6.7: Funciones para operación available

```

static void available_cb(struct lkm_check *check, void*
data){
    struct seq_file *m = data;
    seq_printf(m, "%s\n", check->alias);
}
static int available_show(struct seq_file *m, void *v){
    core_for_each_available(available_cb, m);
    return 0;
}
static int available_open(struct inode *inode, struct file
*file){
    return single_open(file, available_show, NULL);
}
static const struct file_operations fops_available = {
    .owner = THIS_MODULE,
    .open = available_open,
    .read = seq_read,
    .llseek = seq_lseek,
    .release = single_release,
};

```

Si se observan los parámetros para `core_for_each_available()`, uno de ellos es la función `available_cb()`, que ilustra el mecanismo de desacoplamiento implementado para mantener en `core_debugfs.c` la presentación al usuario y en `core.c` la lógica interna. En vez de delegar al núcleo la tarea de mostrar los contenidos de los nodos que componen las listas que recorre, la responsabilidad se mantiene en `core_debugfs.c`, el cual define una función de *callback* (“retrollamada”), en este caso `available_cb`, que muestra por pantalla con `seq_printf()` el alias de los *checks* disponibles, y que `core_for_each_available()` ejecutará por cada nodo de la lista.

De esta forma, el núcleo (`core.c`) permanece completamente desacoplado e independiente del mecanismo de presentación (`seq_file`), permitiendo reutilizar su lógica o incluso migrar a otros sistemas de archivos sin depender de `debugfs`.

También, se puede ver cómo las funciones de lectura o desplazamiento del cursor son `seq_read` y `seq_lseek`, actuando como intermediarias entre el VFS y la función `show()`, invocando tantas veces esta última como sea necesario y gestionando los buffers devueltos al espacio de usuario.

6.1.10. Asignación de Memoria en Kernel

Para reservar memoria de forma dinámica en el Kernel, no se recurre a funciones de espacio de usuario como `malloc()` para reservar, `realloc()` para redimensionar, o `free()` para liberar [28, 8. Kernel Memory Allocation for Module Authors – Part 1]. En su lugar, se recurre a funciones específicas que reservan la memoria directamente en el espacio de kernel. Así, pueden considerarse análogas `kmalloc()`, `krealloc()`, y `kfree()` (con diferencias por configuraciones específicas de kernel).

Reserva de memoria en este TFG

En este proyecto, existen tres casos principales referentes a la gestión de memoria dinámica, empleando `kzalloc()`, `kcalloc()`, y `kfree()` [46].

En primer lugar, la reserva de entradas para las listas. Ya sea crear un `entry_available` cuando un nuevo *check* está disponible, o un `entry_selected` cuando se selecciona un *check*, será necesario reservar de forma dinámica la memoria para la entrada de la lista. Para ello, se recurrirá a la función `kzalloc()`, que devuelve un puntero a memoria inicializada a cero.

Después, hay partes del código donde se ejecuta la funcionalidad deseada de todos los *check* seleccionados, como en `core_for_each_selected()`. En este caso, se crea un doble puntero a `lkm_check` (un array de punteros a `lkm_check`) y se le reserva memoria con capacidad para el número de elementos en `list_selected` mediante `kcalloc`, que reserva memoria para arrays y la inicializa a cero. A continuación, se recorre la lista `list_selected` y se extraen los `lkm_check`, almacenándolos en dicho array. Para cada elemento, se llama a `try_module_get()` con el objetivo de incrementar el `refcount` del módulo propietario y evitar que este sea descargado mientras se ejecuta su código, previniendo así situaciones de *use-after-free*. Una vez construido este *snapshot*, que desacopla iteración y ejecución, se libera el mutex asociado a la lista y se ejecutan los *check* sin mantener bloqueos, lo que evita problemas de contención o posibles deadlocks. Tras la ejecución de cada *check*, se invoca `module_put()` para restaurar `refcount` a su estado previo.

Por último, una vez un *check* se descarga (y por ende un `entry_available` se debe eliminar), un `entry_selected` se deselecta, o los array mencionados previamente terminan su función, se liberará la memoria asociada con `kfree()`.

6.1.11. Devolución de errores en Kernel

Mientras que en espacio de usuario es posible finalizar una función incluso cuando se devuelven errores, en el kernel prima no únicamente seguir una convención de valores de error, sino además garantizar que se deshace la actividad previa al error para evitar pánicos en el sistema.

Para ello, por un lado, existen macros predefinidas para diferentes errores en `errno-base.h` del código fuente del kernel [47]. En este proyecto, por ejemplo, se emplean `-ENOMEM` para gestionar que una reserva de memoria para un nodo de las listas no pudo hacerse con éxito, `-EEXIST` para cuando un elemento no ha sido encontrado, `-ENODEV` cuando se comprueba si `debugfs` está activado en el sistema, o `-EFAULT` cuando se intenta obtener por consola texto que haya pasado el usuario.

Por otro lado, vinculadas a estas macros existen las funciones `IS_ERR()`, que comprueba si el valor devuelto es un error, `ERR_PTR()`, que convierte un número de error a un puntero que lo representa, y `PTR_ERR()`, que convierte un puntero que se sospecha que es un error de vuelta al código de error numérico [43, 2. Error handling]. También, siguiendo el estilo de programación de kernel de Linux, si una función desarrollada devuelve un `int`, en caso de error devolverá los códigos de error directamente; si, en su lugar, devuelve un puntero, deberá devolver un puntero a error mediante `ERR_PTR()` en caso de error.

7

Desarrollo de un *Check* personalizado

La fortaleza de esta herramienta es la capacidad de los usuarios de crear sus propios *checks*, pero para ello es necesario seguir unas pautas específicas que desemboquen en un LKM personalizado compatible. En este capítulo se ilustrarán los requisitos a cumplir mediante el desarrollo del LKM `check_b`, que contará los procesos en el sistema en el momento de ejecución.

7.1. Código mínimo en `.c` necesario

Los ítems específicos para el código en C del LKM son:

- Licencias SPDX para GPL y autoría de copyright.
- Cabecera ABI del proyecto desarrollado: `/include/lkm_check.h`
- Cabeceras disponibles por el kernel: `linux/init.h`, `linux/module.h`
- Definición de `struct lkm_check` estático (`static`) con campos completados de acuerdo a ABI `lkm_check.h`. Esto incluye versión de ABI; módulo propietario del struct; nombre, alias, y categoría del *check*; y función principal a ejecutar.
- Función de inicialización del LKM que invoque `core_register_check()`.
- Función de finalización del LKM que invoque `core_unregister_check()`.
- Función de capacidad principal del *check*. Implementa el comportamiento que se desee emular.

- Sigüientes macros de metadatos de LKM como mínimo:
 - Licencia: `MODULE_LICENSE()`.
 - Autor: `MODULE_AUTHOR()`.
 - Descripción: `MODULE_DESCRIPTION()`.
 - Alias: `MODULE_ALIAS()`. Debe ser igual al campo `.alias` del struct definido.

Tomando el ejemplo de desarrollo del `check_b.c`, la primera parte será entonces, obligatoriamente, de acuerdo a la licencia del proyecto, un identificador SPDX para GPL, preferiblemente también incluyendo información de contacto del copyright original, como en el fragmento 6.2.

En el fragmento 7.1 se muestra el siguiente paso. Se incluirá la cabecera de `/include/lkm_check.h`, que define cómo es un *check* en esta herramienta, `linux/init.h`, cabecera con las macros para inicializar módulos y funciones (`__init`, `__exit`) y `linux/module.h`, con toda la información necesaria para poder compilar al código en un LKM y que el kernel pueda reconocerlo como tal. Adicionalmente, como la funcionalidad deseada es contar procesos y compartirlo a espacio de usuario, se empleará `linux/sched/signal.h` y `linux/printk.h` respectivamente. Después, se crea un `struct lkm_check` estático (`static`) para asegurar que ningún otro archivo pueda modificar sus contenidos. En esta estructura se completarán los campos de acuerdo a lo definido en `/include/lkm_check.h`, por lo que versión de ABI será la macro contenida en `lkm_check.h`, el propietario deberá ser `.owner = THIS_MODULE` para que el propio struct posea una referencia a sí mismo, nombre, alias, y categoría, y por último en `.run` la función que devuelva `int` e implemente la funcionalidad principal del *check*.

Código 7.1: `check_b`: cabeceras y struct `lkm_check`

```
#include <linux/init.h>
#include <linux/module.h>
#include <linux/printk.h>
#include <linux/sched/signal.h>
#include "lkm_check.h"

static struct lkm_check check_b = {
    .abi_version = LKM_CHECK_ABI_VERSION,
    .owner = THIS_MODULE,
    .name = "check_b",
    .alias = "check_b",
    .category = "sample",
    .run = check_b_enumeration,
};
```

Una vez terminadas estas definiciones iniciales, se comenzará con las funciones a implementar. En la sección 6.1 se indicaba cómo el kernel ejecuta una función de

inicialización y una de salida cuando el módulo se carga y se descarga, respectivamente. Así, se definirán dos funciones: `static int __init nombre_función_init(void)` y `static void __exit nombre_función_exit(void)` que se añadirán en las macros `module_init()` y `module_exit()`. Para este *check*, se optará por nombres sencillos: `check_init(void)` y `check_exit(void)`. De acuerdo al framework, estas funciones deben, como mínimo, invocar a las funciones de registro (en `init`) y desregistro (en `exit`) al núcleo. La definición completa, incluyendo prototipos de función antes del `struct lkm_check` y las macros necesarias (`__init`, `module_init()`, `__exit`, `module_exit()`) se muestra en el fragmento 7.2.

Código 7.2: `check_b`: funciones `__init` y `__exit`

```
static int __init check_init(void);
static void __exit check_exit(void);

static int __init check_init(void){
    return core_register_check(&check_b_check);
}
module_init(check_init);

static void __exit check_exit(void){
    core_unregister_check(&check_b_check);
}
module_exit(check_exit);
```

La funcionalidad principal de este módulo se almacenará en el campo `.run` de la estructura `check_b`, como muestra 7.3. En este caso, se desea contar el número de procesos en el sistema, por lo que se creará una función con nombre `check_b_enumeration()`, se asignará al campo `.run()` del `struct`, y se añadirá su prototipo con el resto de funciones.

Código 7.3: `check_b`: funcionalidad de enumeración de procesos

```
static int check_b_enumeration(struct seq_file *m);

static struct lkm_check check_b_check = {
    [...]
    .run = check_b_enumeration,
};

static int check_b_enumeration(struct seq_file *m){
    struct task_struct *task;
    int count = 0;
    rcu_read_lock();
    for_each_process(task)
        count++;
}
```

```
rcu_read_unlock();
seq_printf(m,
    "___Check_%s___\n"
    "___Total_processes:%d\n", check_b_check.alias,
    count);
return 0;
}
```

Finalmente, se añadirán las macros de metadatos de los LKMs. Según este framework: licencia, autor, alias, y descripción. Con todos pasos aplicados, el *check* final estará disponible en el apéndice en [B.1](#).

7.2. Código mínimo del Kbuild necesario

Los ítems mínimos a cumplir según el framework para el archivo Kbuild son:

- Licencias SPDX para GPL y autoría de copyright.
- Configuración `ccflags` para incluir en compilación a la carpeta del proyecto `include/` con cabeceras.

Siguiendo el ejemplo de desarrollo de `check_b`, un ejemplo completo se muestra en [7.4](#).

Código 7.4: Código mínimo kbuild para ejemplo `check_b`

```
# SPDX-License-Identifier: GPL-2.0
# Copyright (C) 2026 Saul Fernandez Garcia
<https://github.com/saulfernandezgarcia>

# Build sfgcheck_b
obj-m := sfgcheck_b.o
sfgcheck_b-objs = check_b.o
ccflags-y := -I$(src)/.././include
```

Tras el identificador SPDX e información de copyright, se crea una variable `obj-m` que indica la compilación a un módulo LKM. A esta se le asigna el valor de un objeto `sfgcheck_b.o` que almacenará todos los objetos que conformarán el módulo; esto permite que, en el caso de que el desarrollo del *check* abarque varios archivos, todos ellos se puedan agregar a un único módulo. Después, se añaden los objetos que componen el LKM (en este caso, únicamente `check_b.o`, que se crea para `check_b.c`) a través del campo `sfgcheck_b-objs`. Por último, se especifica que durante la compilación es necesario incluir las cabeceras contenidas en `/include` mediante la variable `ccflags`.

8

Evaluación

En este capítulo se evaluará la herramienta primero haciendo un uso guiado de esta y posteriormente identificando sus limitaciones.

8.1. Ejemplo de ciclo de vida

Para ilustrar un uso de inicio a fin de la herramienta desarrollada, se explicará un caso de inserción del núcleo, inserción, selección, ejecución y retirada de dos *checks*, y cierre completo del núcleo.

8.1.1. Recomendaciones y entorno

La herramienta desarrollada devuelve información al buffer de logs de nivel de kernel y al espacio de usuario (mediante `seq_file`). Por ello, para poder visualizar los nuevos mensajes en tiempo real, se recurrirá a la herramienta `journalctl` con `sudo journalctl -dmesg -f`.

Para poder invocar acciones sobre los *checks* desarrollados, será necesario acceder al directorio de debugfs que contiene los archivos virtuales vinculados. En este proyecto, se encuentran en `/sys/kernel/debug/lkmsfg`.

Por último, en el proyecto se recomienda la inserción de los módulos mediante `modprobe` ya que este aprovecha los alias creados para LKMs y sus rutas. No obstante, también se puede recurrir a `insmod` para insertar módulos conociendo su ruta. Durante el desarrollo de LKMs, se almacenan en `/lib/modules/[_kernel_version_]`. Para este proyecto, los módulos, una vez compilados, se guardan en las siguientes rutas, donde la primera contiene únicamente al núcleo, y la segunda contiene en subcarpetas a los *checks* compilados.

```

saul@ubuntu-lkm:~/develop/kernel/lkm_sfg$ sudo make mic
[sudo] password for saul:
make -C /lib/modules/6.8.0-111-generic/build M=/home/saul/develop/kernel/lkm_sfg modules
make[1]: Entering directory '/usr/src/linux-headers-6.8.0-111-generic'
warning: the compiler differs from the one used to build the kernel
The kernel was built by: x86_64-linux-gnu-gcc-12 (Ubuntu 12.3.0-1ubuntu1-22.04.3) 12.3.0
You are using: gcc-12 (Ubuntu 12.3.0-1ubuntu1-22.04.3) 12.3.0
CC [M] /home/saul/develop/kernel/lkm_sfg/core/core.o
CC [M] /home/saul/develop/kernel/lkm_sfg/core/core_debugfs.o
LD [M] /home/saul/develop/kernel/lkm_sfg/core/sfgcore.o
CC [M] /home/saul/develop/kernel/lkm_sfg/checks/check_a/check_a.o
LD [M] /home/saul/develop/kernel/lkm_sfg/checks/check_a/sfgcheck_a.o
CC [M] /home/saul/develop/kernel/lkm_sfg/checks/check_b/check_b.o
LD [M] /home/saul/develop/kernel/lkm_sfg/checks/check_b/sfgcheck_b.o
MODPOST /home/saul/develop/kernel/lkm_sfg/Module.symvers
CC [M] /home/saul/develop/kernel/lkm_sfg/core/sfgcore.mod.o
LD [M] /home/saul/develop/kernel/lkm_sfg/core/sfgcore.ko
BTF [M] /home/saul/develop/kernel/lkm_sfg/core/sfgcore.ko
Skipping BTF generation for /home/saul/develop/kernel/lkm_sfg/core/sfgcore.ko due to unavailability of vmlinux
CC [M] /home/saul/develop/kernel/lkm_sfg/checks/check_a/sfgcheck_a.mod.o
LD [M] /home/saul/develop/kernel/lkm_sfg/checks/check_a/sfgcheck_a.ko
BTF [M] /home/saul/develop/kernel/lkm_sfg/checks/check_a/sfgcheck_a.ko
Skipping BTF generation for /home/saul/develop/kernel/lkm_sfg/checks/check_a/sfgcheck_a.ko due to unavailability of vmlinux
CC [M] /home/saul/develop/kernel/lkm_sfg/checks/check_b/sfgcheck_b.mod.o
LD [M] /home/saul/develop/kernel/lkm_sfg/checks/check_b/sfgcheck_b.ko
BTF [M] /home/saul/develop/kernel/lkm_sfg/checks/check_b/sfgcheck_b.ko
Skipping BTF generation for /home/saul/develop/kernel/lkm_sfg/checks/check_b/sfgcheck_b.ko due to unavailability of vmlinux
make[1]: Leaving directory '/usr/src/linux-headers-6.8.0-111-generic'
make -C /lib/modules/6.8.0-111-generic/build M=/home/saul/develop/kernel/lkm_sfg INSTALL_MOD_DIR=sfg modules_install
make[1]: Entering directory '/usr/src/linux-headers-6.8.0-111-generic'
INSTALL /lib/modules/6.8.0-111-generic/sfg/core/sfgcore.ko
SIGN /lib/modules/6.8.0-111-generic/sfg/core/sfgcore.ko
INSTALL /lib/modules/6.8.0-111-generic/sfg/checks/check_a/sfgcheck_a.ko
SIGN /lib/modules/6.8.0-111-generic/sfg/checks/check_a/sfgcheck_a.ko
INSTALL /lib/modules/6.8.0-111-generic/sfg/checks/check_b/sfgcheck_b.ko
SIGN /lib/modules/6.8.0-111-generic/sfg/checks/check_b/sfgcheck_b.ko
DEPMOD /lib/modules/6.8.0-111-generic
Warning: modules_install: missing 'System.map' file. Skipping depmod.
make[1]: Leaving directory '/usr/src/linux-headers-6.8.0-111-generic'
depmod -a
make -C /lib/modules/6.8.0-111-generic/build M=/home/saul/develop/kernel/lkm_sfg clean
make[1]: Entering directory '/usr/src/linux-headers-6.8.0-111-generic'
CLEAN /home/saul/develop/kernel/lkm_sfg/Module.symvers
make[1]: Leaving directory '/usr/src/linux-headers-6.8.0-111-generic'
saul@ubuntu-lkm:~/develop/kernel/lkm_sfg$

```

Figura 8.1: Puesta a prueba: compilación del núcleo y *checks*.

```

/lib/modules/$(uname -r)/sfg/core
/lib/modules/$(uname -r)/sfg/checks

```

8.1.2. Ejemplo de uso

Este caso práctico girará alrededor del núcleo `sfgcore` y los *checks* `check_a` y `check_b`. El código fuente estará disponible en el repositorio del proyecto [48].

En primer lugar, se compila el proyecto. Entre los comandos `make` desarrollados, se llamará a `make mic`, como se ve en la figura 8.1, que invocará la compilación (m - make), la instalación (i - install) y la limpieza de archivos generados en el proceso (c - clean). Realizar `clean` al final es apropiado ya que se limpian los archivos generados en la carpeta del proyecto sin borran los `.ko` generados al instalarse en la ruta del sistema `/lib/modules/kernel_version/sfg`.

En la imagen se ha resaltado el comando `depmod -a`, que facilita la generación de dependencias entre los módulos compilados y cualquier otro ya presente.

La figura 8.2 muestra la ruta de instalación de los archivos `.ko` generados para este sistema operativo y versión de kernel: `/lib/modules/6.8.0-111-generic/sfg`.

Como también se observa en la misma imagen, se han generado tres archivos `.ko`: `sfgcore.ko`, `sfgcheck_a.ko`, `sfgcheck_b.ko`. Si se ejecuta `modinfo` sobre estos archivos, se obtendrán más detalles como el alias o autor, como se ve

```

saúl@ubuntu-lkm:/lib/modules/6.8.0-111-generic/sfg$
saúl@ubuntu-lkm:/lib/modules/6.8.0-111-generic/sfg$ ls
checks  core
saúl@ubuntu-lkm:/lib/modules/6.8.0-111-generic/sfg$ ls core
sfgcore.ko
saúl@ubuntu-lkm:/lib/modules/6.8.0-111-generic/sfg$ ls checks
check_a  check_b
saúl@ubuntu-lkm:/lib/modules/6.8.0-111-generic/sfg$ ls checks/check_a; ls checks/check_b
sfgcheck_a.ko
sfgcheck_b.ko
saúl@ubuntu-lkm:/lib/modules/6.8.0-111-generic/sfg$

```

Figura 8.2: Puesta a prueba: ubicación de archivos `.ko` tras compilación.

```

saúl@ubuntu-lkm:/lib/modules/6.8.0-111-generic/sfg$ modinfo core/sfgcore.ko
filename:       /lib/modules/6.8.0-111-generic/sfg/core/sfgcore.ko
description:    Development version of core for lkm management.
author:        SAUL FERNANDEZ GARCIA
alias:         sfgcore
license:       GPL
srcversion:    E4A86E6A991E21E78B162D5
depends:
retpoline:     Y
name:          sfgcore
vermagic:      6.8.0-111-generic SMP preempt mod_unload modversions
saúl@ubuntu-lkm:/lib/modules/6.8.0-111-generic/sfg$ modinfo checks/check_a/sfgcheck_a.ko
filename:       /lib/modules/6.8.0-111-generic/sfg/checks/check_a/sfgcheck_a.ko
description:    Sample check plugin for console printing.
author:        SAUL FERNANDEZ GARCIA
alias:         check_a
license:       GPL
srcversion:    7434DDE45508DBB60BD868D
depends:        sfgcore
retpoline:     Y
name:          sfgcheck_a
vermagic:      6.8.0-111-generic SMP preempt mod_unload modversions
saúl@ubuntu-lkm:/lib/modules/6.8.0-111-generic/sfg$ modinfo checks/check_b/sfgcheck_b.ko
filename:       /lib/modules/6.8.0-111-generic/sfg/checks/check_b/sfgcheck_b.ko
description:    Sample check plugin for process enumeration
author:        SAUL FERNANDEZ GARCIA
alias:         check_b
license:       GPL
srcversion:    878EC8DC36F0D5D070028C9
depends:        sfgcore
retpoline:     Y
name:          sfgcheck_b
vermagic:      6.8.0-111-generic SMP preempt mod_unload modversions

```

Figura 8.3: Puesta a prueba: `modinfo` de núcleo y *checks* de ejemplo.

en la figura 8.3.

Después, desde cualquier ruta, se insertan los módulos en el kernel mediante `sudo modprobe [alias del check]` (recuérdese que se emplea `sudo` por los requisitos de privilegio del kernel). La imagen 8.4 muestra una consola ejecutando `journalctl` para visualizar el estado de la inserción de los LKMs compilados y la propia inserción de estos.

En la captura 8.5 se muestra el caso de que se intente insertar un *check* antes que el núcleo. Debido al mapeo de dependencias, se insertará primero el núcleo y después el *check*.

Una vez el núcleo comienza a insertarse, entre los logs generados, como muestra la captura 8.6, aparece uno indicando que los archivos virtuales han sido creados. Estos se podrán encontrar navegando a la ruta `/sys/kernel/debug/lkmsfg`, la cual requiere permisos de superusuario para acceder.

```

saul@ubuntu-lkm:~/desktop$ sudo modprobe sfgcore
saul@ubuntu-lkm:~/desktop$ sudo modprobe check_a
saul@ubuntu-lkm:~/desktop$ sudo modprobe check_b
saul@ubuntu-lkm:~/desktop$

saul@ubuntu-lkm:~$ sudo journalctl --dmesg -f
mai 09 12:08:11 ubuntu-lkm kernel: kauditd_printk_skb: 1 callbacks suppressed
mai 09 12:08:11 ubuntu-lkm kernel: audit: type=1400 audit(1778321291.324:93): a
pparmor="DENIED" operation="capable" class="cap" profile="/usr/sbin/cupsd" pid=
9540 comm="cupsd" capability=12 capname="net_admin"
mai 09 12:08:20 ubuntu-lkm kernel: 10:08:20.915405 timesync vgsvcTimeSyncWorker
: Radical guest time change: 70 799 659 419 000ns (GuestNow=1 778 321 300 915 3
75 000 ns GuestLast=1 778 250 501 255 956 000 ns fSetTimeLastLoop=true)
mai 09 12:59:18 ubuntu-lkm kernel: lkm CORE: loading into kernel
mai 09 12:59:18 ubuntu-lkm kernel: lkm CORE: creating debugfs directory
mai 09 12:59:18 ubuntu-lkm kernel: lkm CORE: directory was created
mai 09 12:59:18 ubuntu-lkm kernel: lkm CORE: creating interactive command files
mai 09 13:00:20 ubuntu-lkm kernel: lkm CORE: removing from kernel
mai 09 13:00:20 ubuntu-lkm kernel: lkm CORE: removed from kernel
mai 09 13:00:28 ubuntu-lkm kernel: audit: type=1326 audit(1778324428.340:94): a
uid=1000 uid=1000 gid=1000 ses=3 subj=snap-store.ubuntu-software pid=2076
comm="pool-org.gnome." exe="/snap/snap-store/1216/usr/bin/snap-store" sig=0 arc
h=00000000 syscall=93 compat=0 ip=0x7bdc9193b08 code=0x50000
mai 09 13:00:57 ubuntu-lkm kernel: lkm CORE: loading into kernel
mai 09 13:00:57 ubuntu-lkm kernel: lkm CORE: creating debugfs directory
mai 09 13:00:57 ubuntu-lkm kernel: lkm CORE: directory was created
mai 09 13:00:57 ubuntu-lkm kernel: lkm CORE: creating interactive command files
mai 09 13:01:19 ubuntu-lkm kernel: lkm: check check_a requesting registration
mai 09 13:01:19 ubuntu-lkm kernel: lkm: check check_a began registration
mai 09 13:01:19 ubuntu-lkm kernel: lkm: check check_a finished registration
mai 09 13:01:23 ubuntu-lkm kernel: lkm: check check_b requesting registration
mai 09 13:01:23 ubuntu-lkm kernel: lkm: check check_b began registration
mai 09 13:01:23 ubuntu-lkm kernel: lkm: check check_b finished registration

```

Figura 8.4: Puesta a prueba: invocación de `modprobe` sobre núcleo y *checks* para inserción y visualización de mensajes por buffer de kernel.

```

saul@ubuntu-lkm:~/desktop$ date; sudo modprobe check_a
sam. 09 mai 2026 13:08:42 CEST
saul@ubuntu-lkm:~/desktop$

mai 09 13:08:42 ubuntu-lkm kernel: lkm CORE: loading into kernel
mai 09 13:08:42 ubuntu-lkm kernel: lkm CORE: creating debugfs directory
mai 09 13:08:42 ubuntu-lkm kernel: lkm CORE: directory was created
mai 09 13:08:42 ubuntu-lkm kernel: lkm CORE: creating interactive command files
mai 09 13:08:42 ubuntu-lkm kernel: lkm: check check_a requesting registration
mai 09 13:08:42 ubuntu-lkm kernel: lkm: check check_a began registration
mai 09 13:08:42 ubuntu-lkm kernel: lkm: check check_a finished registration

```

Figura 8.5: Puesta a prueba: invocación de `modprobe` sobre un *check* antes que el núcleo.

Si se comprueban los *checks* disponibles mediante una lectura del archivo `available`, se devuelven tanto `check_a` como `check_b`. Como todavía no ha sido ninguno seleccionado, `selected` consecuentemente no devuelve información. Esto se ve en la figura 8.7.

En la figura 8.8 se añaden los *checks* `check_a` y `check_b` escribiendo al archivo virtual `add` mediante el comando `echo`. Una comprobación con `cat selected` permite ver que efectivamente están seleccionados (y, por lo tanto, en la lista `list_selected`).

Una vez seleccionados, se comanda la obtención de resultados con `results`, que invocará la función `run` de cada *check*, viéndose los resultados en la imagen 8.9. El primer *check*, `check_a`, simplemente imprime en espacio de usuario una oración y en logs a nivel de kernel otra. `check_b` cuenta el número de procesos en el dispositivo y los imprime por espacio de usuario, así como un saludo desde logs de kernel.

Con la ejecución completada de los *checks*, en la imagen 8.10 se ve su deselección pasando al archivo virtual `remove` los nombres de los *checks*.

Antes de retirar del kernel los *checks* y el núcleo, se aprovechará este caso para mostrar cómo seleccionar y deseleccionar todos los *checks* disponibles en un solo comando en la imagen 8.11. Primero, escribiendo cualquier mensaje a `addall` se comanda la selección de todo *check* en `list_available`. De forma complementaria, escribiendo a `empty` se vacía la lista de seleccionados.

Los LKMs de los *checks* se podrán retirar del kernel siempre y cuando no estén en la lista de seleccionados. Con esta comprobación realizada, la imagen

```
root@ubuntu-lkm:/sys/kernel/debug# cd lkmsfg
root@ubuntu-lkm:/sys/kernel/debug/lkmsfg# ls
add  addall  available  empty  remove  results  selected
root@ubuntu-lkm:/sys/kernel/debug/lkmsfg#
```

Figura 8.6: Puesta a prueba: comprobación con `ls` de archivos virtuales creados en debugfs.

```
root@ubuntu-lkm:/sys/kernel/debug/lkmsfg# cat available
check_a
check_b
root@ubuntu-lkm:/sys/kernel/debug/lkmsfg# cat selected
root@ubuntu-lkm:/sys/kernel/debug/lkmsfg#
```

Figura 8.7: Puesta a prueba: uso de `available` para visualizar *checks* disponibles.

8.12 enseña la invocación de `modprobe -r` con el nombre de cada *check*. Nótese que cuando se retira el último *check*, será acompañado por la retirada del núcleo, finalizando así un ciclo de vida completo de la herramienta.

```

root@ubuntu-lkn:/sys/kernel/debug/lknsfsg# echo check_a > add
root@ubuntu-lkn:/sys/kernel/debug/lknsfsg# echo check_b > add
root@ubuntu-lkn:/sys/kernel/debug/lknsfsg# cat selected
check_a
check_b
root@ubuntu-lkn:/sys/kernel/debug/lknsfsg#

```

```

mai 09 14:03:37 ubuntu-lkn kernel: lkn: plugin check_a was not in selected list
. It will now be added.
mai 09 14:03:37 ubuntu-lkn kernel: lkn: added to 'selected' the check with alia
s: check_a
mai 09 14:06:24 ubuntu-lkn kernel: lkn: plugin check_b was not in selected list
. It will now be added.
mai 09 14:06:24 ubuntu-lkn kernel: lkn: added to 'selected' the check with alia
s: check_b

```

Figura 8.8: Puesta a prueba: uso de **add** para añadir *checks* y **selected** para comprobar su selección.

```

root@ubuntu-lkn:/sys/kernel/debug/lknsfsg# cat results
==== check_a ====
--- Check A is running its specific code!

==== check_b ====
--- Check check_b ---
- Total processes:232
root@ubuntu-lkn:/sys/kernel/debug/lknsfsg#

```

```

mai 09 14:10:52 ubuntu-lkn kernel: Check A is saying hi!
mai 09 14:10:52 ubuntu-lkn kernel: Check B is saying hi!

```

Figura 8.9: Puesta a prueba: uso de **results** para obtención de resultados de ejecución de funcionalidad de los *checks* seleccionados.

```

root@ubuntu-lkn:/sys/kernel/debug/lknsfsg# echo check_a > remove
root@ubuntu-lkn:/sys/kernel/debug/lknsfsg# echo check_b > remove
root@ubuntu-lkn:/sys/kernel/debug/lknsfsg# cat selected
check_a
check_b
root@ubuntu-lkn:/sys/kernel/debug/lknsfsg#

```

```

mai 09 14:20:15 ubuntu-lkn kernel: lkn: removed from 'selected' the check with
alias: check_a
mai 09 14:20:21 ubuntu-lkn kernel: lkn: removed from 'selected' the check with
alias: check_b

```

Figura 8.10: Puesta a prueba: uso de **remove** para la deselección de *checks* de la lista de seleccionados.

```

root@ubuntu-lkn:/sys/kernel/debug/lknsfsg# echo 1 > addall
root@ubuntu-lkn:/sys/kernel/debug/lknsfsg# cat selected
check_a
check_b
root@ubuntu-lkn:/sys/kernel/debug/lknsfsg# echo 1 > empty
root@ubuntu-lkn:/sys/kernel/debug/lknsfsg# cat selected
root@ubuntu-lkn:/sys/kernel/debug/lknsfsg#

```

```

mai 09 14:27:21 ubuntu-lkn kernel: lkn: added to 'selected' the check with alia
s: check_a
mai 09 14:27:21 ubuntu-lkn kernel: lkn: added to 'selected' the check with alia
s: check_b

```

Figura 8.11: Puesta a prueba: uso de **addall** y **empty** para seleccionar y deseleccionar todos los *checks* disponibles.

```

root@ubuntu-lkn:/sys/kernel/debug/lknsfsg# date; lsmod | grep sfg
sun. 09 mai 2026 14:40:23 CEST
sfgcheck_b      12288  0
sfgcheck_a      12288  0
sfgcore         24576  2 sfgcheck_a,sfgcheck_b
root@ubuntu-lkn:/sys/kernel/debug/lknsfsg# modprobe -r check_a check_b
root@ubuntu-lkn:/sys/kernel/debug/lknsfsg# date; lsmod | grep sfg
sun. 09 mai 2026 14:40:35 CEST
root@ubuntu-lkn:/sys/kernel/debug/lknsfsg#

```

```

mai 09 14:40:30 ubuntu-lkn kernel: lkn: check check_a requesting unregistration
mai 09 14:40:30 ubuntu-lkn kernel: lkn: check check_a began unregistration
mai 09 14:40:30 ubuntu-lkn kernel: lkn: check check_a finished unregistration
mai 09 14:40:30 ubuntu-lkn kernel: lkn: check check_b requesting unregistration
mai 09 14:40:30 ubuntu-lkn kernel: lkn: check check_b began unregistration
mai 09 14:40:30 ubuntu-lkn kernel: lkn: check check_b finished unregistration
mai 09 14:40:30 ubuntu-lkn kernel: lkn CORE: removing from kernel
mai 09 14:40:30 ubuntu-lkn kernel: lkn CORE: removed from kernel

```

Figura 8.12: Puesta a prueba: retirada de kernel de LKMs *core* y *checks*.

9

Limitaciones

En cuanto a desarrollo, en primer lugar, actualmente sigue habiendo cierto nivel de acoplamiento entre el núcleo y los *checks* cuando se comanda la ejecución de la funcionalidad de estos. Específicamente, en `core.c` se crea un doble puntero a `lkm_check` en vez de delegar el conocimiento de la lógica interna y de los *checks* únicamente a la capa intermediaria y de presentación que establece debugfs.

Por otro lado, cuando se desarrolla un *check* con esta ABI, se debe incluir campos, como el alias, tanto en la macro de metadatos `MODULE_ALIAS()` como en el campo de `.alias` del `struct lkm_check`, lo cual es redundante. Pese a que un desarrollador podría crear un *check* con un alias para el archivo `.ko` como tal, y otro alias distinto para referirse al *check* en el programa, esta segregación para un mismo alias podría originar problemas de consistencia. Además, no existe un proceso automatizado actual que permita imponer el requisito de tener valores idénticos.

En la línea de la inserción de módulos, aunque se puede pasar a los archivos virtuales creados los contenidos de otros archivos para interactuar con los *checks* ya insertados (como, por ejemplo, un documento que contenga un listado de *checks* mediante `cat list.txt > add`), no se dispone actualmente de una forma rápida y automatizada de insertar en el kernel todos los *checks* compilados con la herramienta, y su inserción debe ser manual mediante `insmod` o `modprobe`. También, la ABI define una macro para la versión de esta bajo la que se compiló un *check*; sin embargo, no se han definido todavía controles dentro del propio núcleo para aplicar comprobaciones y no insertar versiones incompatibles.

Finalmente, las funciones de devolución de información a nivel de kernel son todas de tipo informativo y mediante `pr_info()`, sin categorizar en diferentes niveles la seriedad o impacto de los eventos durante la gestión de errores empleando otras llamadas como `pr_err()` para errores.

10

Conclusiones y trabajos futuros

Este capítulo detallará las conclusiones derivadas del TFG y propuestas a posibles trabajos futuros.

10.1. Conclusiones

El proyecto ha cumplido su objetivo principal: facilitar la creación y gestión de comportamientos ejecutados en espacio de kernel mediante una herramienta modular orientada a apoyar la investigación y detección de malware en Linux.

La naturaleza modular del kernel de Linux permite desarrollar y cargar *Loadable Kernel Modules* (LKMs) externos de forma dinámica. La solución propuesta aprovecha esta característica y la amplifica mediante una arquitectura que abstrae y estandariza el desarrollo de módulos, incorporando además mecanismos de control y orquestación de su ejecución dentro del framework. Tal y como se ha demostrado con los *checks* implementados, los módulos desarrollados no tienen por qué limitarse a la emulación de comportamientos maliciosos. Por ello, la herramienta también presenta potencial de aplicación en otros ámbitos relacionados con el desarrollo y análisis de componentes del kernel de Linux donde sea necesario un control minucioso sobre LKMs externos.

Asimismo, ante la ausencia de soluciones que proporcionen una capa estandarizada de gestión y ejecución de LKMs, este trabajo pretende servir como un primer esfuerzo sobre el que puedan construirse futuras herramientas, investigaciones, o contribuciones.

En este sentido, la publicación del proyecto como software de código abierto busca fomentar la colaboración y facilitar su evolución dentro del ecosistema de investigación y desarrollo en Linux.

10.2. Trabajos Futuros

Más allá de solventar los obstáculos identificados en el capítulo 9, el desarrollo de una herramienta basada en esencia y motivación en el kernel de Linux abre múltiples posibilidades para trabajos futuros.

10.2.1. Arquitectura del núcleo

Entre las mejoras más inmediatas, un posible objetivo sería estudiar si desacoplar más el núcleo del conocimiento interno de los *checks* en la ABI `lkm_check.h` y viceversa es algo conveniente o si el nivel actual de desacoplamiento es efectivo para una mirada a largo plazo.

Igualmente, evaluar formas de generar scripts para la automatización de inserción en kernel de todos los LKMs compilados bajo la herramienta y probar diferentes métodos para imponer los ítems definidos en el capítulo 7 para el desarrollo de estos. Como aspecto de mejora en cuanto a *logging* de información, una clasificación más específica de aquello que se devuelve por mensajes de kernel, desde errores y eventos críticos a logs informativos.

Además, segregar la lógica de manipulación de las listas en otro componente del núcleo, como podría ser un archivo `core_lists.c`, podría despejar las responsabilidades del *core* y mantenerlo más como un verdadero gestor que como un gestor y actor en el proceso.

Los nodos de las listas, por otro lado, podrían ver sus campos ampliados para así poder generar estadísticas que devolver al usuario. Campos de interés podrían ser sello de tiempo en el que el *check* se hizo disponible o fue seleccionado, cuándo fue ejecutado por última vez, duración de la última ejecución del *check*, o veces ejecutado desde que fue insertado en el kernel. Estas estadísticas, de misma manera, podrían obtenerse del propio rendimiento del núcleo y así estudiar cuellos de botella.

10.2.2. Ejecución de *checks*

La ejecución de los *checks* seleccionados actualmente se hace mediante un array `snapshot` que, junto con otros controles, evita potenciales *use-after-free*. Sería de interés estudiar la viabilidad de una ejecución de funcionalidad de los *checks* sin bloquear los procesos, similar a un planteamiento *fire-and-forget* mediante un *pool* de hilos de ejecución trabajadores y un hilo que escuche los resultados que estos devuelvan.

10.2.3. Interfaz de usuario

Respecto a la capa de presentación al usuario, se podría explorar nuevas interfaces, más allá de debugfs, que puedan equilibrar esfuerzos de desarrollo y facilidad de uso de la herramienta por parte del usuario. Todo esto, también,

bajo un alineamiento del flujo de trabajo en fases, teniendo una rama desarrollo, una de testeo, y una de producción. Esto se acompañaría de un *test harness* que permitiese implementar tests unitarios o de otro tipo y así facilitar la detección de errores en el código.

10.2.4. Compilación y configuración

En cuanto a la compilación, el esquema actual se basa en Makefile y Kbuild. No obstante, se podría facilitar la gestión de todos los elementos a compilar mediante los menús configurables de `kconfig` [49].

10.2.5. ABI y retrocompatibilidad

La ABI, como se indicó en el capítulo 9, define una macro para versionado pero esta no es empleada por ningún componente del núcleo todavía para gestionar el flujo de ejecución y permitir o no a un *check* ser tratado por el núcleo, por lo que un trabajo a futuro sería implementar dichos controles. Similarmente, con el fin de poder emplear LKMs para *checks* hechos con versiones anteriores de la ABI en caso de que esta crezca, investigar métodos de retrocompatibilidad y así tener una ABI y núcleo resiliente ante cambios.

10.2.6. Documentación y estandarización

Finalmente, para poder hacer de este proyecto uno que pueda ser considerado por la comunidad del kernel de Linux en el futuro, la generación de documentación siguiendo el estilo oficial y la armonización del estilo de código a nuevos estándares marcados por la comunidad.

Bibliografía

- [1] D. Baluta, O. Purdila, V. Badoiu, and dragosargint, “Linux source code layout.” [Online]. Available: <https://linux-kernel-labs.github.io/refs/heads/master/lectures/intro.html#linux-source-code-layout>
- [2] R. Love, *Linux Kernel Development, Third Edition*. Addison-Wesley Professional, 2010, 1. Introduction to the Linux Kernel, 17. Devices and Modules. [Online]. Available: <https://learning.oreilly.com/library/view/linux-kernel-development/9780768696974/ch01.html>
- [3] T. kernel development community, “The linux kernel.” [Online]. Available: <https://docs.kernel.org/>
- [4] D. P. Siewiorek and R. S. Swarz, *Reliable Computer Systems: Design and Evaluation*. CRC Press, 2019.
- [5] J. Stühn, J.-N. Hilgert, and M. Lambertz, “The hidden threat: Analysis of linux rootkit techniques and limitations of current detection tools,” *Digital Threats*, vol. 5, no. 3, Oct. 2024. [Online]. Available: <https://doi.org/10.1145/3688808>
- [6] M. ATT&CK, “Technique t1014: Rootkit,” May 2017. [Online]. Available: <https://attack.mitre.org/versions/v18/techniques/T1014/>
- [7] M. Nadim, W. Lee, and D. Akopian, “Kernel-level rootkit detection, prevention and behavior profiling: A taxonomy and survey,” pp. 3–7, 2023. [Online]. Available: <https://arxiv.org/abs/2304.00473>
- [8] M. ATT&CK, “Hijack execution flow: Dynamic linker hijacking, sub-technique t1574.006,” Mar 2020. [Online]. Available: <https://attack.mitre.org/versions/v18/techniques/T1574/006/>
- [9] M. Alessia, “Analysis and testing of ebpF attack surfaces,” Master’s thesis, Politecnico di Torino, 2024. [Online]. Available: <https://webthesis.biblio.polito.it/33909/>
- [10] H. Xu, W. Du, and S. Chapin, “Detecting exploit code execution in loadable kernel modules,” in *20th Annual Computer Security Applications Conference*, 2004, pp. 101–110.
- [11] P. J. Salzman, M. Burian, O. Pomerantz, B. Mottram, and J. Huang, *The Linux Kernel Module Programming Guide*, December 2022, ch. 1.7 Before We Begin - Modversioning.
- [12] J. Levine, J. Grizzard, and H. Owen, “A methodology to detect and characterize kernel level rootkit exploits involving redirection of the system call table,” Georgia Institute of Technology, Tech. Rep., 2004.
- [13] —, “Detecting and categorizing kernel-level rootkits to aid future detection,” Georgia Institute of Technology, Tech. Rep., 2006.
- [14] M. Landauer, L. Alton, M. Lindorfer, F. Skopik, M. Wurzenberger, and W. Hotwagner, “Trace of the times: Rootkit detection through temporal anomalies in kernel activity,” Ph.D. dissertation, Austrian Institute of Technology and Vienna University of Technology, 2025. [Online]. Available: <https://arxiv.org/html/2503.02402v1>

-
- [15] L. Org, “Linux kernel runtime guard.” [Online]. Available: <https://lkg.org/>
 - [16] M. Schmidt, L. Baumgärtner, P. Graubner, D. Bock, and B. Freisleben, “Malware detection and kernel rootkit prevention in cloud computing environments,” 03 2011, pp. 603 – 610.
 - [17] R. Sisto, “Detection and mitigation of eBPF security risks in the linux kernel,” Master’s thesis, Politecnico di Torino, 2025.
 - [18] K. Billimoria, “Linux kernel programming, second edition - code repository.” [Online]. Available: <https://github.com/PacktPublishing/Linux-Kernel-Programming>
 - [19] M. Domsch and G. Lerhaupt, “Dynamic kernel module support: From theory to practice,” in *Proceedings of the Linux Symposium*, July 2004, vol. Volume One, dell Linux Engineering.
 - [20] L. Rice, *Learning EBPF*. O’Reilly Media, 2023. [Online]. Available: <https://books.google.es/books?id=dW-yEAAAQBAJ>
 - [21] AppArmor, “Apparmor / apparmor.” [Online]. Available: <https://gitlab.com/apparmor/apparmor>
 - [22] SELinuxProject, “Selinuxproject/selinux.” [Online]. Available: <https://github.com/SELinuxProject/selinux>
 - [23] C. Wright, C. Cowan, J. Morris, S. Smalley, and G. Kroah-Hartman, “Linux security module framework,” in *Proceedings of the Ottawa Linux Symposium*, 07 2002, <https://www.kernel.org/doc/ols/2002/ols2002-pages-604-617.pdf>.
 - [24] S. Smalley, C. Vance, and T. Fraser, “Linux security modules: General security hooks for linux.” [Online]. Available: <https://www.kernel.org/doc/html/v6.8/security/lsm.html>
 - [25] T. kernel development community, “Linux security module usage.” [Online]. Available: <https://docs.kernel.org/admin-guide/LSM/index.html>
 - [26] L. K. Documentation, “Linux kernel makefiles,” 2025. [Online]. Available: <https://www.kernel.org/doc/Documentation/kbuild/makefiles.txt>
 - [27] —, “Linux kernel makefiles,” 2025. [Online]. Available: <https://www.kernel.org/doc/Documentation/kbuild/modules.txt>
 - [28] K. Billimoria, *Linux Kernel Programming*, second edition ed. Packt Publishing, February 2024.
 - [29] bootlin and The Linux Kernel, “Gpl-2.0 - licenses/preferred/gpl-2.0 - linux source code v6.8 - bootlin elixir cross referencer,” v6.8. [Online]. Available: <https://elixir.bootlin.com/linux/v6.8/source/LICENSES/preferred/GPL-2.0>
 - [30] T. kernel development community, “Linux kernel licensing rules, 6.8.” [Online]. Available: <https://www.kernel.org/doc/html/v6.8/process/license-rules.html>
 - [31] —, “Linux kernel licensing rules, 7.0.0.” [Online]. Available: <https://docs.kernel.org/process/license-rules.html>
 - [32] I. Free Software Foundation, “The gnu make manual,” 2023. [Online]. Available: <https://www.gnu.org/software/make/manual/make.html>
 - [33] bootlin and The Linux Kernel, “module.h - include/linux/module.h - linux source code v6.8 - bootlin elixir cross referencer,” v6.8. [Online]. Available: <https://elixir.bootlin.com/linux/v6.8/source/include/linux/module.h>
 - [34] —, “module.h - include/linux/module.h - linux source code v6.8 - bootlin elixir cross referencer,” v6.8. [Online]. Available: <https://elixir.bootlin.com/linux/v6.8/source/kernel/module/main.c>

- [35] T. kernel development community, “Driver basics.” [Online]. Available: <https://docs.kernel.org/driver-api/basics.html>
- [36] O. Community and O. Corporation, “Linkedlist - openjdk/jdk.” [Online]. Available: <https://github.com/openjdk/jdk/blob/master/src/java.base/share/classes/java/util/LinkedList.java>
- [37] bootlin and The Linux Kernel, “types.h - include/linux/types.h - linux source code v6.8 - bootlin elixir cross referencer,” v6.8. [Online]. Available: <https://elixir.bootlin.com/linux/v6.8/source/include/linux/types.h>
- [38] —, “list.h - include/linux/list.h - linux source code v6.8 - bootlin elixir cross referencer,” v6.8. [Online]. Available: <https://elixir.bootlin.com/linux/v6.8/source/include/linux/list.h>
- [39] T. kernel development community, “Linked lists in linux.” [Online]. Available: <https://docs.kernel.org/core-api/list.html>
- [40] —, “What is rcu? – “read, copy, update”.” [Online]. Available: <https://docs.kernel.org/RCU/whatisRCU.html>
- [41] —, “Debugfs.” [Online]. Available: <https://docs.kernel.org/filesystems/debugfs.html>
- [42] —, “Overview of the linux virtual file system.” [Online]. Available: <https://docs.kernel.org/filesystems/vfs.html>
- [43] J. Madiou, *Linux Device Driver Development*, second edition ed. Packt Publishing, 2022.
- [44] bootlin and The Linux Kernel, “fs.h - include/linux/fs.h - linux source code v6.8 - bootlin elixir cross referencer,” v6.8. [Online]. Available: <https://elixir.bootlin.com/linux/v6.8/source/include/linux/fs.h>
- [45] T. kernel development community, “The seq_file interface.” [Online]. Available: https://docs.kernel.org/filesystems/seq_file.html
- [46] —, “Memory management apis.” [Online]. Available: <https://www.kernel.org/doc/html/v6.8/core-api/mm-api.html>
- [47] bootlin and The Linux Kernel, “errno-base.h - include/uapi/asm-generic/errno-base.h - linux source code v6.8 - bootlin elixir cross referencer,” v6.8. [Online]. Available: <https://elixir.bootlin.com/linux/v6.8/source/include/uapi/asm-generic/errno-base.h>
- [48] Fernández García, Saúl, “saulfernandezgarcia/lkm_sfg,” 2026, fecha de presentación del código: Junio 2026. [Online]. Available: https://github.com/saulfernandezgarcia/lkm_sfg
- [49] T. kernel development community, “Kconfig language, 6.8.” [Online]. Available: <https://www.kernel.org/doc/html/v6.8/kbuild/kconfig-language.html>
- [50] D. A. Patterson and J. L. Hennessy, *Computer Organization and Design MIPS Edition*, 6th ed. Morgan Kaufmann, 2021.

Apéndice



Estructura del Proyecto

Este capítulo documenta la organización del proyecto a nivel de archivos y carpetas, la relación entre componentes y el flujo de información.

A.0.1. Organización

El proyecto, albergado en `lkm_sfg`, se estructura en los siguientes ítems, como visualiza el diagrama [A.1](#):

- **Carpeta `checks/`:** almacena subcarpetas con los **checks** desarrollados de acuerdo al framework.
- **Carpeta `core/`:** almacena `core.c` con la lógica pura de gestión de *checks* mediante listas, `core_debugfs.c` para permitir al usuario interactuar con el núcleo, y `core_internal.h`, que expone las funciones del *core* a otros recursos como `core_debugfs.c`. Por último, incluye un archivo Kbuild que estructura la compilación y creación del LKM del núcleo (llamado `sfgcore`).
- **Carpeta `include/`:** contiene la ABI para la definición de *checks* “`lkm_check.h`”.
- **Makefile:** comanda la compilación del proyecto e instalación de módulos.
- **Kbuild:** una vez se ejecuta el Makefile, este archivo especifica qué rutas se van a compilar y convertir en LKMs (de acuerdo a los Kbuild contenidos en cada subcarpeta). Como mínimo, se creará el LKM para el núcleo.
- **LICENSE.txt:** declaración de GPL-2.0.
- **README.md:** pequeño documento introductorio a la herramienta.

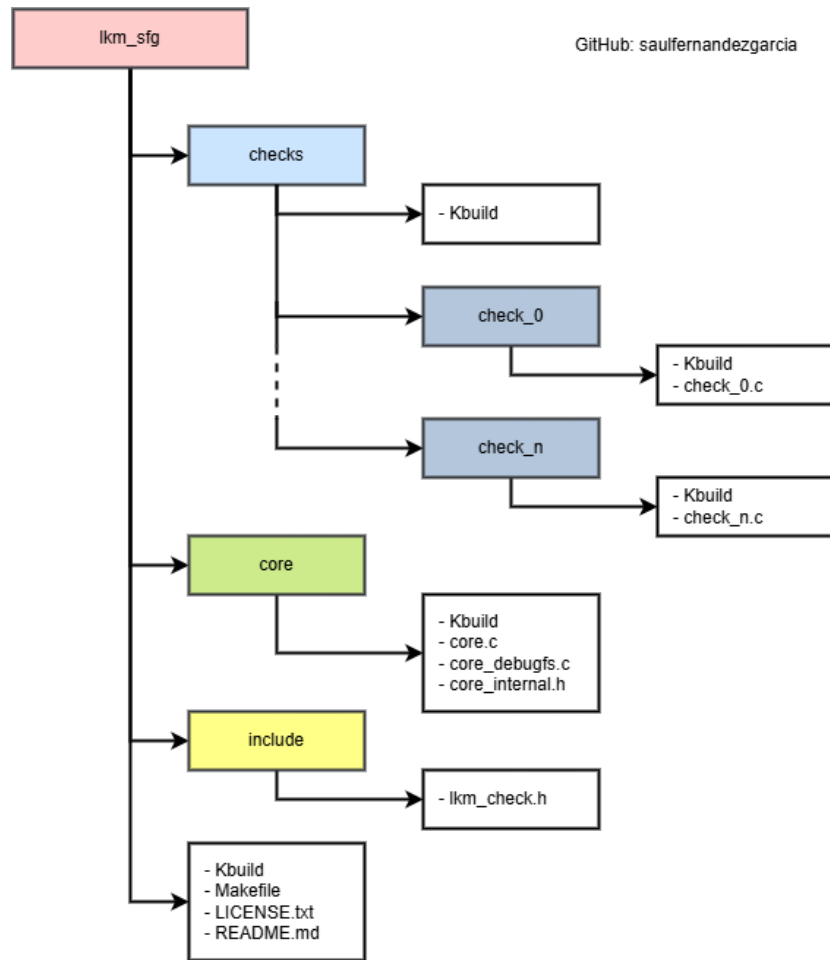


Figura A.1: Estructura del proyecto desarrollado

El contenido de cada *check* dentro de *checks* dependerá del autor de este. No obstante, contendrá como mínimo un archivo Kbuild para la compilación del código en un LKM y uno o varios archivos en C que lo conformen.

A.0.2. Flujo lógico de información

En la figura A.2 se puede comprobar la relación lógica entre los elementos del proyecto.

A primera vista, se distinguen dos grandes grupos: *Backend* y *User Interface*, conteniendo los elementos de la lógica interna y de la interfaz de usuario, respectivamente.

Backend es la lógica interna de gestión. Se muestra la relación de *core.c* hacia la gestión de las listas y creación de nodos de estas (*entry_[selected | available]*). También se incluye aquí la ABI *lkm_check.h* para la definición de *checks*, sobre la cual el núcleo no tiene interacción directa (reflejando la intención de separación de responsabilidades que se buscaba con este proyecto). Por último, *core_internal.h* es el nexo entre el núcleo y la interfaz de usuario.

User Interface es la parte a través la cual el usuario interactúa con los *checks*. *core_debugfs.c* recurre a las funciones de manipulación de listas que expone *core_internal.h*, único elemento

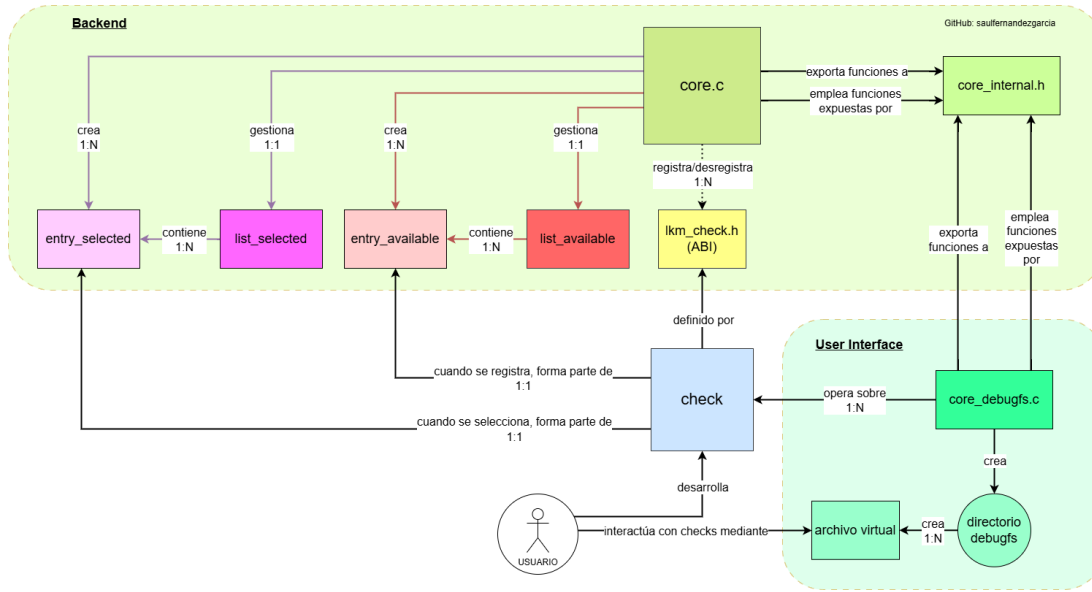


Figura A.2: Flujo lógico de información

del *backend* que tiene accesible. `core_debugfs.c` a su vez crea el directorio de debugfs y los archivos virtuales correspondientes para operar sobre los *check*.

Finalmente, se visualiza la relación de quien emplea el programa, tanto como desarrollador de los *check* como usuario de estos comandando su ejecución mediante los archivos virtuales de `core_debugfs.c`.

A.0.3. Desarrollo de propia infraestructura de plugins en C

Uno de los objetivos del proyecto era que la herramienta fuese modular. Para ello, se optó por crear una infraestructura similar al uso de *plugins*, donde se pueden adherir a un programa principal nuevas capacidades desde programas más pequeños y dependientes a este. La naturaleza modular del kernel de Linux facilitó precisamente este objetivo.

La compilación del proyecto genera un núcleo (*core*) llamado `sfgcore`, que es el programa principal, y los *check* que se hayan definido, que serán los plugins. Tanto el programa principal como los plugins serán objetos `.ko` insertables en el kernel como LKMs externos.

Un plugin y un *check* se refieren a lo mismo: funcionalidad gestionable por el núcleo y estructurada mediante `lkm_check.h`. En este archivo se define un `struct lkm_check` con información clave como el nombre del *check* desarrollado o su funcionalidad (campo `int (*run)` como puntero a una función), y se importan las funciones de registro `core_register_check()` y `core_unregister_check()`, exportadas por el núcleo. No obstante, si el núcleo no está insertado en el kernel o su módulo no existe, no se podrán resolver los símbolos y el sistema rechazará la inserción del *check*. En caso de que sí exista el núcleo pero no esté insertado, la resolución de dependencias hará que el kernel cargue primero el núcleo y, a continuación, el *check*.

Así, un usuario podrá crear su propia funcionalidad mediante *check* de acuerdo a `struct`

`lkm_check` y garantizando que estos se registran y desregistran del núcleo.

A.1. El núcleo del programa: `core`

Toda la lógica que culmina en lo que se define en este proyecto como núcleo, *core*, o gestor, se contiene en esta carpeta. Esta sección elaborará en cada uno de los elementos que la componen.

A.1.1. `core.c`

Como pieza clave de la herramienta, cumple con varias responsabilidades. La primordial es la manipulación e interacción con las listas que almacenan nodos con *checks*. Para ello, existen las listas `list_available` y `list_selected`, albergando structs de tipo `entry_available` y `entry_selected` correspondientemente para almacenar los *checks* disponibles (*available*) y los seleccionados para futura ejecución (*selected*).

Su función de entrada, `core_init`, es el punto de inicio del LKM en cuanto se inserta con éxito en el kernel, y delega en `core_debugfs_init()` la creación del directorio de debugfs y sus archivos virtuales vinculados a funciones de lectura o escritura. De forma complementaria, `core_exit` garantiza retirar del kernel al gestor de forma controlada, asegurando que las listas estén vacías, que no quede memoria que haya sido reservada por el kernel sin liberar, y que el directorio de debugfs es vaciado.

Como se ha mencionado previamente, obligatoriamente, el núcleo tiene que ser capaz de poder registrar o retirar *checks* que así lo soliciten, y define y exporta la implementación de las siguientes funciones, cuyos prototipos están en `include/lkm_check.h`:

- `int core_register_check:` bajo el control del mutex `lock_list_available`, se crea un ejemplar de `entry_selected` que almacena una referencia al *check* registrado y se encola a `list_available`.
- `void core_unregister_check:` bajo el control de los mutexes `lock_list_selected()` y `lock_list_available()`, se busca, retira, y libera la memoria del *check* de la lista de seleccionados si se encuentra, y después se realiza lo mismo de la lista de disponibles. Con el refcount del *check* a retirar reducido a 0 y las listas sin su presencia, se podrá retirar también del kernel de forma segura.

Más allá de los preparativos de inicialización y finalización y registro o retirada de plugins, el núcleo implementa funciones para interactuar con las listas expuestas a través de sus prototipos en `core_internal.h`. Estas son:

- `core_for_each_available()`: realiza la ejecución secuencial de una función de callback que acepta un `lkm_check` sobre cada ítem de la lista `list_available`. Se guardan en un array dinámico (reservado mediante `kcalloc`) referencias a los *checks* y se les aumenta su refcount hasta que son ejecutados, actuando como una imagen del estado de la lista cuando se consultó (un *snapshot*). Así, no se bloquea la lista con el mutex si la callback se alarga y se evitan potenciales *use-after-free* en el período de gracia entre la orden de ejecución y la ejecución.
- `core_for_each_selected()`: realiza la ejecución secuencial de una función de callback que acepta un `lkm_check` sobre cada ítem de la lista `list_selected`. Igualmente, opera con un *snapshot* para no bloquear la lista y evitar errores de *use-after-free*.

- `core_select_check`: añade a la lista `list_selected` un *check* específico. Para ello, comprueba que esté en la lista de disponibles antes de añadirlo.
- `core_addall`: añade a la lista `list_selected` todos los *checks* disponibles.
- `core_remove_check`: retira y libera de `list_selected` un ítem específico.
- `core_empty_selected`: vacía `list_selected`, liberando a su vez memoria.

A.1.2. `core_debugfs.c`

Esta es la pieza del programa encargada de exponer el estado de este vía `debugfs`, convertir estructuras de datos a texto mediante `seq_file`, y gestionar input del usuario para hacer llamadas al núcleo.

Contiene dos funciones principales de preparación y limpieza:

- `core_debugfs_init`: crea el directorio de `debugfs` y archivos virtuales con operaciones específicas asignadas en el código. Recurre a una macro multilínea para sintetizar y facilitar código de creación de los archivos virtuales, en imitación a otros segmentos de código del kernel de Linux, como en `linux/.../rwonce.h` empleado por `linux/list.h`.
- `core_debugfs_exit`: borra el directorio de `debugfs` y sus archivos de forma recursiva.

La ubicación del directorio `debugfs` creado para este proyecto será: `/sys/kernel/debug/lkmsfg`, y los archivos virtuales contenidos serán los siguientes:

- `available`: imprime por consola los *checks* disponibles (que están insertados en el kernel).
- `selected`: imprime por consola los *checks* seleccionados.
- `results`: comanda la ejecución secuencial de los *checks* en orden de cola (FIFO).
- `add`: recibe un string que puede consistir en un solo *check* o una serie de ellos separados por espacios, tabulaciones, o comas, y realiza parsing para incluirlo(s) en la lista de seleccionados.
- `remove`: recibe un string que puede consistir en un solo *check* o una serie de ellos, al igual que con `add`, separados por espacios, tabulaciones, o comas, y los retira de la lista de seleccionados.
- `addall`: añade a la lista de seleccionados a todos los *checks* disponibles.
- `empty`: retira de la lista de seleccionados a todos los *checks*.

Cada archivo virtual se corresponde con un `struct file_operations`, el cual define el módulo al que pertenece (mediante el ítem `.owner = THIS_MODULE`, como sucede con los *checks*), que siempre será el *core*, y las operaciones vinculadas. A continuación, se explicará brevemente el flujo de estos archivos y sus funciones. Nótese que para las funciones de selección o deselección de *checks*, la gestión de errores sigue *best-effort*, por lo que intentará tratar todos los *checks* y no detendrá la ejecución por uno o más que no hayan podido ser tratados.

available

Vinculado al `struct fops_available`, cuando recibe una acción de lectura, como `cat available`, invoca en la función de apertura del campo `.open: available_open`, la cual llama al método `single_open()` con la función `available_show()`, la cual pasa a la función del núcleo `core_for_each_available()` la callback `available_cb()` para visualizar los ítems de la lista de disponibles.

selected

Similar al archivo virtual `available`, está vinculado al `struct fops_selected`. Un comando de lectura, como `cat selected`, desencadenará la cadena de funciones `selected_open()` > `selected_show()` > `core_for_each_selected()`, esta última siendo del *core* y recibiendo la callback `selected_cb` para visualizar ítems seleccionados.

results

Vinculado al `struct fops_results`, una operación de lectura comanda el procesado para la obtención de resultados, y el ítem `.open` con `results_open()`, siguiendo una cadena similar a los otros usos de `single_open()`, llega a invocar `core_for_each_selected()` con la función de callback `results_cb()`, la cual comanda la ejecución de la función `run` de cada *check* en la lista de seleccionados mediante `check->run(m)`.

El objeto que recibe esta función es `seq_file *m`, un puntero a un `seq_file` que se le facilita a la funcionalidad del *check* para poder gestionar la devolución de información al usuario.

add

Vinculado al `struct fops_add`, cuando recibe una acción de escritura que le pasa uno o más *checks* separados por un espacio, coma, o tabulación, como `echo check_a, check_b > add`, invoca su campo `.write: add_write`, que fragmenta la lista de *check* descritos y los pasa de uno en uno al *core* mediante `core_select_check()` para su inclusión.

remove

Vinculado al `struct fops_remove`, sigue la misma línea de acción que el archivo virtual `add`. Tras un comando de escritura como `echo check_b, check_a > remove`, fragmenta la lista de *check* descritos y los pasa de uno en uno al *core* mediante `core_remove_check()` para su retirada de la lista de seleccionados.

addall

Vinculado al `struct fops_addall`, cuando recibe una acción de escritura, como `echo 1 > addall`, invoca su función vinculada en el ítem `.write: addall_write`, que solicita al *core* la selección de todos los *check* disponibles.

empty

Vinculado al `struct fops_empty`, cuando recibe una acción de escritura, como `echo 1 > empty`, invoca su función vinculada en el ítem `.write`, que en este caso es la función `empty_write()`, que inicia el vaciado de la lista de seleccionados mediante `core_empty_selected()`, una de las funciones que expone el *core*.

A.1.3. core_internal.h

Esta cabecera actúa como el intermediario entre el núcleo y el sistema de debugfs. Por un lado, se exponen las funciones del núcleo que este permite a `core_debugfs.c` utilizar para interactuar de forma indirecta con las listas. Igualmente, se exponen las funciones de `core_debugfs.c` para la creación y borrado del directorio debugfs para que el núcleo pueda comandar estas acciones durante su función de inicialización y de salida, respectivamente.

A.2. La ABI del programa: include/lkm_check.h

Una ABI, *Application Binary Interface*, se define como una interfaz a nivel de sistema operativo empleada cuando se programan aplicaciones y que especifica cómo programas compilados llaman a funciones e interactúan con datos [50, Computer Abstractions and Technology - 1.4].

`lkm_check.h` actúa como ABI del proyecto ya que define, por un lado, cómo `struct lkm_check` va a estar organizado en memoria (orden de campos específico), cómo se pasan `check` creados a otras funciones, y cómo programas externos se comunican con el sistema de *checks* mediante las funciones `core_register_check()` y `core_unregister_check()`. Esto permite que incluso con un *core* ya compilado, un usuario pueda desarrollar sus propios plugins siempre y cuando siga la misma ABI que se empleó durante la compilación de dicho *core*.

Los campos contenidos en `struct lkm_check` son:

- `int abi_version`: campo para el versionado del *check* y su control de ejecución. En el caso de que la ABI se vea obligada a crecer, se actualizará el campo de forma acorde.
- `struct module *owner`: `struct` de `linux/module.h` que permite el acceso a información exclusiva de los LKM, como su `refcount` o listas de dependencias de módulos.
- `const char name[PLUGIN_MAX_NAME]`: nombre del *check*.
- `const char category[PLUGIN_MAX_CATEGORY]`: categoría del *check*.
- `const char alias[PLUGIN_MAX_ALIAS]`: alias del *check*.
- `int (*run)(struct seq_file *m)`: función principal del *check*. Cuando se comande la ejecución del plugin, esta se cederá a esta función.

A su vez, incluye los *function prototypes* para las funciones de registro mencionadas anteriormente:

- `int core_register_check(struct lkm_check *check)`: registro.
- `void core_unregister_check(struct lkm_check *check)`: retirada.

Estos son empleados por los *checks* en sus funciones de `__init` y `__exit`, pero no los implementan. Como se mencionó previamente, se exportan desde el núcleo.

A.3. La carpeta `checks` y sus contenidos

Esta carpeta contiene los proyectos para cada *check* desarrollado. Contiene un archivo `Kbuild` donde se puede especificar qué subcarpetas de *checks* se desean compilar en LKMs. Aprovechando la naturaleza recursiva de `Kbuild`, cada subcarpeta deberá incluir su propio *Kbuild* para definir cómo compilar el *check* específico.

B

Código

B.1. *Check* B: check_b

Código B.1: Código completo en C para ejemplo check_b

```
// SPDX-License-Identifier: GPL-2.0
/**
 * Copyright (C) 2026 Saul Fernandez Garcia
 * <https://github.com/saulfernandezgarcia>
 */

#include <linux/init.h>
#include <linux/module.h>
#include <linux/printk.h>
#include <linux/sched/signal.h>

#include "lkm_check.h"

static int check_b_enumeration(struct seq_file *m);
static int __init check_init(void);
static void __exit check_exit(void);

static struct lkm_check check_b = {
    .abi_version = LKM_CHECK_ABI_VERSION,
    .owner = THIS_MODULE,
    .name = "check_b",
    .alias = "check_b",
    .category = "sample",
    .run = check_b_enumeration,
};
```

```
static int check_b_enumeration(struct seq_file *m){
    pr_info("Check_B_is_saying_hi!\n");

    struct task_struct *task;
    int count = 0;

    rcu_read_lock();
    for_each_process(task)
        count++;
    rcu_read_unlock();

    seq_printf(m,
        "____Check_%s____\n"
        "-_Total_processes:%d\n", check_b.alias, count);
    return 0;
}

static int __init check_init(void){
    return core_register_check(&check_b);
}
module_init(check_init);

static void __exit check_exit(void){
    core_unregister_check(&check_b);
}
module_exit(check_exit);

MODULE_LICENSE("GPL");
MODULE_ALIAS("check_b");
MODULE_AUTHOR("SAUL_FERNANDEZ_GARCIA");
MODULE_DESCRIPTION("Sample_check_plugin_for_process_enumeration");
```